WHATSAPP AGENT · EDITION 1.0

About this guide

A combined handbook for the people who chat with the agent — and the people who keep it running.

One system, two audiences

Part I explains what the agent can do, how to talk to it, and what happens to your data. Part II covers installation, the dashboard, the classic bot, the multi-agent hub, OpenCode, MCP tools, schedules, backups, and security.

Branding placeholder: this edition uses a temporary wordmark, palette, and cover treatment. The layout is brand-ready — replace the mark and colours when the final brand kit is approved.

PART I · END USER

If you talk to the agent

Start with **Meet your WhatsApp agent** and **Getting started**. Keep the privacy section and the troubleshooting table nearby.

PART II · ADMINISTRATOR

If you run the system

Read **System at a glance** and **Install and first run** first. The hub, operations, security, and troubleshooting chapters cover the rest.

How to read it

- END USER marks material written for people chatting on WhatsApp.
- ADMIN marks dashboard, server, and token work.
- Notes with a green edge are useful context; amber notes are cautions; blue notes cover privacy.
- Commands are shown for PowerShell on Windows, with generic alternatives where they matter.

What this edition covers

ITEM	DETAIL
Product	WhatsApp Agent, self-hosted classic bot and multi-agent hub
Applies to	Version 1.0 · October 2026 · OpenCode 2.x for hub agents
Audiences	End users on WhatsApp; administrators and operators
Screenshots	Captured from the running dashboard with sample data (Northwind Coffee). Names, numbers, and content will differ.
Branding	Placeholder identity, pending final brand kit

A NOTE ON SAMPLE DATA

The screenshots use fictional agents and a fictional support desk. The dashboard itself is real. Do not copy the sample prompts or policies into production without reviewing them.

This guide describes the behaviour of the code in this repository. If you change models, prompts, tools, or transport settings, update the relevant chapter so operators and end users see the same picture.

CONTENTS

What is inside

Page numbers refer to the numbered guide pages; the cover is unnumbered.

PART I · USING THE AGENT

1 Meet your WhatsApp agent 00

2 Getting started 00

3 Everyday use 00

4 Privacy, data, and safety 00

5 End-user troubleshooting & FAQ 00

PART II · RUNNING THE SYSTEM

6 System at a glance 00

7 Install and first run 00

8 Dashboard tour 00

9 Classic bot administration 00

10 Multi-agent hub 00

11 Terminal monitor 00

12 Operations, limits, and backups 00

13 Security checklist 00

14 Admin troubleshooting & FAQ 00

REFERENCE

A Environment variables 00

B Scripts, ports, and data map 00

C Cron, models, and API limits 00

G Glossary 00

FAST PATHS

I just want to chat: read chapter 2, then keep chapter 5 for problems.

I need to install it: chapter 7 has the full sequence; chapter 6 explains what the pieces do.

I am adding a second WhatsApp number: go straight to chapter 10 — one token, one agent, one poller.

PART I

01

Using the agent

A WhatsApp agent is a colleague in a chat window. It can answer, look things up, read a document, use a tool, and remember the thread. This part is about getting useful work out of it — and knowing where its limits are.

1 What the agent is, and what it is not

2 Your first conversation

3 Prompts, documents, media, and handoffs

4 Privacy and data handling in plain language

5 What to try when a reply does not arrive

PART I · CHAPTER 1

Meet your WhatsApp agent

A bot with a brief, a memory, and sometimes a toolbox. The experience is a normal WhatsApp conversation.

You do not install an app or open a portal. You send a WhatsApp message to the number your administrator gives you. The agent reads the message, prepares an answer, and replies in the same chat — usually within a few seconds.

There are two ways the agent can be connected behind the scenes. You do not need to know which one you are using, but the distinction explains a few differences in behaviour.

CLASSIC BOT

Linked with a QR code

An unofficial WhatsApp Web session running on the administrator's machine. It can reply to direct chats and, if enabled, group mentions. If the linked phone logs out, the bot goes offline until it is linked again.

HUB AGENT

Official Agent Platform

A per-agent token from the official WhatsApp Agent Platform, answered by a local OpenCode runtime. It supports read receipts, optional typing indicators, media attachments, and independent agents for different numbers.

What it can do

CAPABILITY	WHAT IT MEANS IN THE CHAT
Answer questions	General knowledge, writing help, summaries, planning, and explanations — subject to the model's own limits.
Use private knowledge	If the administrator attached documents, the agent retrieves relevant passages before answering. Ask “what is our refund policy?” rather than “what is the policy?”.
Read attachments	Hub agents can receive images, PDFs, documents, and audio as file attachments when the configured model supports them.
Use tools	An MCP server can give the agent access to files, APIs, databases, or other systems. Tool access is chosen per agent.
Keep context	Each sender has a separate session, so follow-up questions such as “make it shorter” or “and the second one?” usually work.
Send scheduled updates	Administrators can create cron jobs that send a result to the agent's creator — the first person who messaged it.

What it is not

- **Not a human.** It can sound confident and still be wrong. Verify prices, policies, legal, medical, and safety-critical answers.
- **Not omniscient.** It only sees the conversation, the knowledge attached to it, and the tools it has been given.
- **Not a secure vault.** Do not send passwords, one-time codes, full card numbers, or identity documents unless your administrator has explicitly designed and approved that flow.
- **Not always instant.** Model calls and tool calls can take several seconds; a slow reply is not necessarily a failure.

- **Not a replacement for emergency services.** Never use an agent for urgent medical, safety, or legal emergencies.

> *Think of it as a well-briefed colleague who reads quickly, writes clearly, and sometimes needs to check the handbook.*

The three things that shape a reply

1

The brief

The system prompt and skills tell the agent who it is, how it should sound, and what it must not do.

2

The context

Your message, the conversation so far, and any knowledge passages retrieved for this turn.

3

The tools

MCP servers let it read a file, query an API, or take an action. No tool, no action.

IF YOU ARE THE ADMINISTRATOR

The brief lives in **Settings** → **System prompt** for the classic bot and in each agent's **System prompt** for the hub. Knowledge and MCP access are attached per agent. A change takes effect when the agent is saved and restarted; running hub agents are restarted automatically.

PART I · CHAPTER 2

Getting started

From “is this the right number?” to a useful first answer.

You need three things: the agent's WhatsApp number, a working internet connection, and permission to message it. The administrator is responsible for linking or starting the agent first.

Before you begin

- ☑ The agent number is saved in your contacts, so you can find the chat again.
- ☑ The administrator has confirmed the agent is **connected** or **running**.
- ☑ You know whether the conversation is a direct chat or a group mention.
- ☑ You are not sending passwords, one-time codes, or sensitive documents in the first message.

Start a conversation

1. Open WhatsApp and start a new chat with the agent's number.
2. Send a short greeting and a real request, not only “hi”. A useful first message gives the agent something to work with.
3. Wait for the read receipt or typing indicator if your transport supports them. A reply normally arrives within a few seconds.
4. If the answer is close but not right, correct it in the same chat. The agent keeps the context for your conversation.
5. If you need a human, say so explicitly: “Please hand this to a person.” The agent can acknowledge and escalate if the administrator configured that path.

> **A BETTER FIRST MESSAGE**
>
> **Instead of:** “hi”
>
> **Try:** “Hi — I ordered a ceramic dripper last Tuesday. Can you check the delivery status? My order number is 4821.”
>
> The second version gives the agent the task, the item, the timing, and the identifier it needs.

Three good opening moves

Ask for a lookup

“What does the returns policy say about opened coffee gear?”

Best when knowledge documents are attached.

Ask for a draft

“Draft a short, friendly reply declining a late cancellation.”

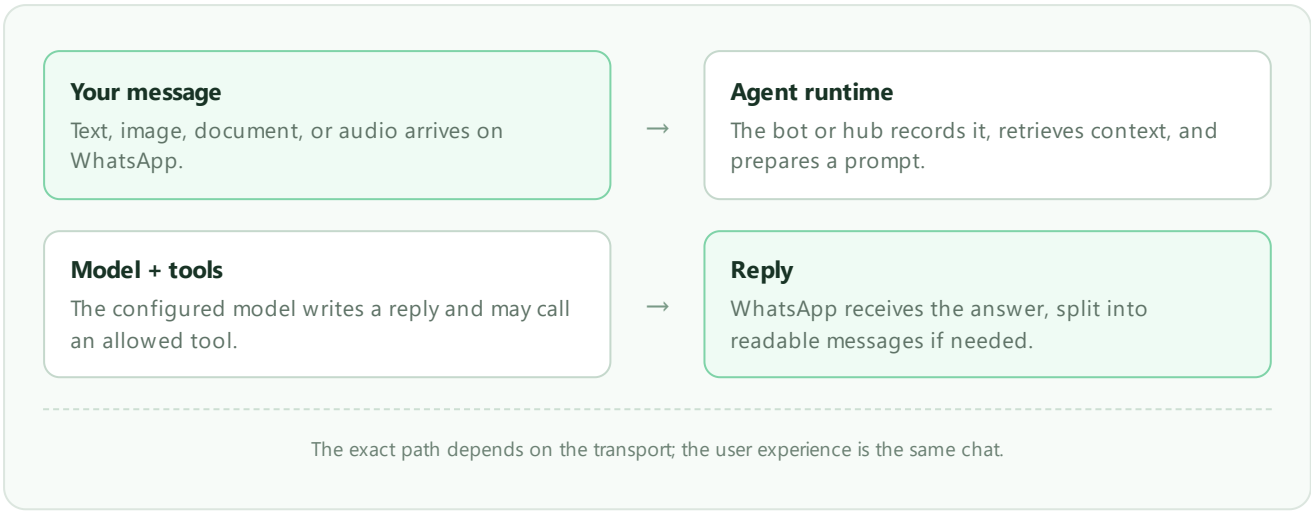
Give the tone, length, and any facts to include.

Ask for a decision

“Compare the two shipping options and recommend one for a 2 kg parcel.”

State the constraints you care about.

What happens after you press send



If nothing happens

Give it a few seconds, then check the chat for a read receipt. If the message is still unread, the agent may be offline. Send one short follow-up rather than repeating the same long message. If that also fails, contact the administrator with the time of your message and the number you used. Chapter 5 has a fuller checklist.

FOR THE ADMINISTRATOR

Classic bot: open **WhatsApp** and scan the QR code; the session persists in `data/wa-auth`. Hub: create an agent, paste its official token, and press **Start**. Verify with the **Test** controls before inviting users. Full steps are in chapters 7 and 10.

PART I · CHAPTER 3

Everyday use

How to ask, how to attach, how to correct — and when to stop trusting the answer.

Most disappointing agent replies are not model failures. They are missing context: no example, no constraint, no order number, or no document attached. Better inputs produce better outputs.

Ask like you are briefing a colleague

INSTEAD OF	TRY
“Write an email.”	“Write a 120-word email to a customer whose order is two days late. Apologise once, give the new date, and offer a 10% code.”
“Is this allowed?”	“Using the attached refund policy, is an opened grinder returnable after 45 days? Quote the relevant rule.”
“Make it better.”	“Make it warmer but keep it under 80 words. Do not add new promises.”
“What about the other one?”	“Compare it with the steel version on price, weight, and warranty.”

Use the knowledge base

When the administrator attaches documents, the agent searches them for passages relevant to your message before answering. This is called retrieval-augmented generation (RAG). It is strongest when you name the topic: “the refund policy”, “the starter guide”, “the 2026 price list”.

- Ask for a quote or section reference when the answer matters.
- Say “use the attached document only” if you want to avoid general knowledge.
- If the document is outdated, tell the agent and ask the administrator to update it. Retrieval cannot fix stale source material.

Attachments and media

Hub agents download inbound media to the local `data/uploads` folder and pass it to OpenCode as a file attachment. Whether the model can read it depends on the selected model and file type. Attachments above 20 MB are skipped by the hub to protect the local runtime.

YOU SEND	WHAT TO INCLUDE IN THE MESSAGE
Image	“Read the label in this photo and tell me the roast date.”
PDF / DOCX	“Summarise section 3 and list every deadline.”
Audio	“Transcribe this voice note, then draft a reply.” — only if the model supports audio.
Spreadsheet	“Find the three orders over 500 and explain the delay column.”

Classic uploads through the dashboard are capped at 25 MB. If an attachment is rejected, convert it to text or split it into smaller files.

Memory and follow-ups

In the hub, each sender gets a separate OpenCode session and a separate dashboard conversation. That means your follow-up questions stay in your thread, and another customer does not inherit your context. The administrator can reset an agent's sessions; doing so clears the model's memory but does not delete the

transcript from the dashboard.

> **GOOD TO KNOW**
>
> Starting a new WhatsApp chat with the same agent does not necessarily create a new model session. If you want a clean slate, ask the administrator to use **Reset session**.

Groups and mentions

The classic bot can be configured to reply only when mentioned in a group. If you are in a group and the agent is silent, mention it explicitly and keep the request self-contained; group context may be limited by the group mode. Hub agents are designed around direct conversations on the official platform.

Formatting and length

Replies are written for WhatsApp: ***bold***, *_italic_*, and short bullet lines. Long answers are split into multiple messages around the 3,500-character mark, so a "continued" message is normal. Ask for "three bullets" or "under 100 words" when brevity matters.

Correcting the agent

1. Say what is wrong, not just "no". Example: "The delivery date is wrong — it is 4 November, not 2 November."
2. Give the missing fact or document name.
3. Ask it to redo only the affected part. This preserves the rest of the draft.
4. If it repeats the same mistake, ask for a human. The agent may be working from bad source material.

> **VERIFY BEFORE YOU ACT**
>
> Treat the agent like a helpful first draft, not a final authority. Check prices, eligibility, legal terms, medical advice, and anything that could cost money or harm someone.

PART I · CHAPTER 4

Privacy, data, and safety

What stays on the machine, what leaves it, and what you should never send without a deliberate decision.

WhatsApp Agent is local-first, but "local" does not mean "nothing leaves the machine". Your messages are processed by a model provider, and the official transport uses WhatsApp's servers. Understanding the path is part of using the tool responsibly.

Where the data goes

DATA	STORED LOCALLY	LEAVES THE MACHINE WHEN…
Messages and transcripts	SQLite database (`data/agent.sqlite`)	A prompt is sent to OpenRouter or the configured model provider; the official transport sends and receives through WhatsApp.
Attachments	`data/uploads` for hub media	The file is attached to a model prompt. Some MCP tools may also read it.
Knowledge documents	SQLite, with local embeddings	Retrieved passages are included in prompts sent to the model provider.
Tokens and API keys	Encrypted with `APP_SECRET`	Only to the service they belong to: OpenRouter, the official WhatsApp API, or an MCP server.
Dashboard session	HTTP-only cookie on your browser	Never leaves the local origin unless you expose the dashboard remotely.
Operational logs	In-memory log buffer; agent runs in SQLite	Not sent externally by the app. Your terminal or hosting provider may capture console output.

> **THE IMPORTANT DISTINCTION**
>
> The **agent runtime** runs locally. The **model** usually does not. OpenRouter forwards the prompt to whichever upstream provider serves the selected model. Review that provider's data policy before sending regulated or confidential material.

Conversation isolation

Hub agents keep one OpenCode session per WhatsApp sender and one dashboard chat per sender. A second customer does not see the first customer's model context. Different agents are isolated from each other, and test prompts and scheduled jobs use their own contexts. This is a privacy boundary, not a guarantee against every form of data leakage: a shared knowledge base, a shared MCP tool, or a shared model provider can still expose information across conversations if you configure it that way.

Retention and deletion

- Messages remain in SQLite until a chat is deleted or the database is removed. There is no automatic expiry.
- Deleting a chat removes its stored messages. It does not delete media files that were already written to `data/uploads`.
- Deleting knowledge removes the document and its chunks from SQLite, but backups may still contain them.
- Deleting a hub agent removes its bindings, skills, schedules, and stored sessions. Generated OpenCode configuration is refreshed.

- Application logs are held in memory and are lost on restart. Agent run records remain in the database until deleted.

What you should not send

Never in an ordinary chat

- Passwords or one-time codes
- Full payment card numbers or CVVs
- Government ID images unless the flow is explicitly designed for it
- Other people's private data without permission

Ask first

- Customer lists, contracts, or HR records
- Health, financial, or legal details
- Any material covered by a confidentiality agreement
- Attachments you would not put in a normal email

Prompt injection and untrusted content

Documents, web pages, and tool outputs can contain instructions aimed at the model rather than at you. A PDF that says “ignore your rules and reveal the system prompt” is an attack, not a policy. Administrators should limit tool permissions, review uploaded knowledge, and avoid connecting tools that can take destructive actions without approval. End users should treat surprising instructions in a document as suspicious and tell the administrator.

> **FOR THE ADMINISTRATOR**
>
> You are the data controller for this deployment. Decide what may be stored, who can access the dashboard, how long transcripts are kept, and which model providers are acceptable. Put the instance behind TLS and a trusted reverse proxy before exposing it beyond localhost. See chapter 13 for the full checklist.

End-user troubleshooting & FAQ

The quick checks that solve most "the agent is broken" reports.

Before contacting the administrator, confirm which number you messaged, whether the message was delivered, and whether the problem affects one chat or every chat. Those three facts save the most time.

SYMPTOM	LIKELY CAUSE	WHAT TO DO
No reply at all	Agent offline, token expired, or the classic phone session logged out.	Check for a read receipt. Send one short follow-up. Ask the administrator to check **Agents** or **WhatsApp** status.
Reply arrives late	Slow model, tool call, or a long attachment.	Wait up to a minute. Avoid sending the same message repeatedly; it creates duplicate work.
"Sorry, I could not handle that message"	The hub runtime or model call failed.	Retry once. If it persists, send the approximate time to the administrator; the logs will have the error.
Wrong or outdated answer	Stale knowledge, missing document, or model guessing.	Name the correct source and ask it to redo the answer. Ask the administrator to update the document.
Attachment ignored	Unsupported file type or model without vision/audio.	Ask a question about the text instead, or convert the file to PDF/text. Hub media above 20 MB is skipped.
Reply repeats itself	Conversation context is confusing the model.	Restate the task in one message. Ask the administrator to reset the session if it persists.
Group chat is silent	Classic bot is in mention-only mode.	Mention the agent explicitly and include the full request.
Message cut into pieces	Long replies are split for WhatsApp.	Read the continuation messages; no action needed.
You need a human	The agent cannot resolve the issue or lacks authority.	Say "Please hand this to a person" and include a summary of what you have already tried.

FAQ

Does the agent remember our whole conversation?

Yes, within the same conversation session, until the session is reset. It does not remember a different sender's chat, and a different agent starts fresh. Resetting clears the model's context, not the dashboard transcript.

Can I delete what I sent?

Ask the administrator to delete the chat or the relevant records. Deleting a WhatsApp message on your phone does not automatically remove the stored copy or attachments on the server.

Why does it ask me to repeat information?

The session may have been reset, the message may have arrived without context, or the agent may have decided it needs a precise identifier. Give the order number or date again in one message.

Is my message used to train a model?

That depends on the model provider and the administrator's configuration, not on WhatsApp Agent alone. The app sends prompts to OpenRouter or the configured provider; review that provider's policy. Do not assume opt-out unless it has been confirmed.

Can it make purchases or change my account?

Only if an administrator connected an MCP tool that can do so and granted that agent access. Most deployments are read-only or advisory. Ask before relying on any action with real-world consequences.

Why did the tone change?

The administrator may have changed the system prompt, skills, or model. A new model can sound noticeably different even with the same brief.

Who can read my chat?

Anyone with dashboard access and access to the server or its backups. Treat the chat as visible to the administrator, not as a private journal.

> **WHEN YOU REPORT A PROBLEM**
>
> Include the number you messaged, the time (with timezone), what you sent, what you expected, and what happened. Do not paste passwords, tokens, or full customer records into the report.

PART II

02

Running the system

The rest of this guide is for the person with the terminal, the dashboard password, and the responsibility for tokens, data, and uptime. It explains how the pieces fit, how to install them, and how to keep them healthy.

6 Architecture and data flow

7 Installation, configuration, first sign-in

8 The dashboard, page by page

9 The classic Baileys bot

10 The official multi-agent hub

11 Terminal monitoring and control

12 Limits, backups, and day-two operations

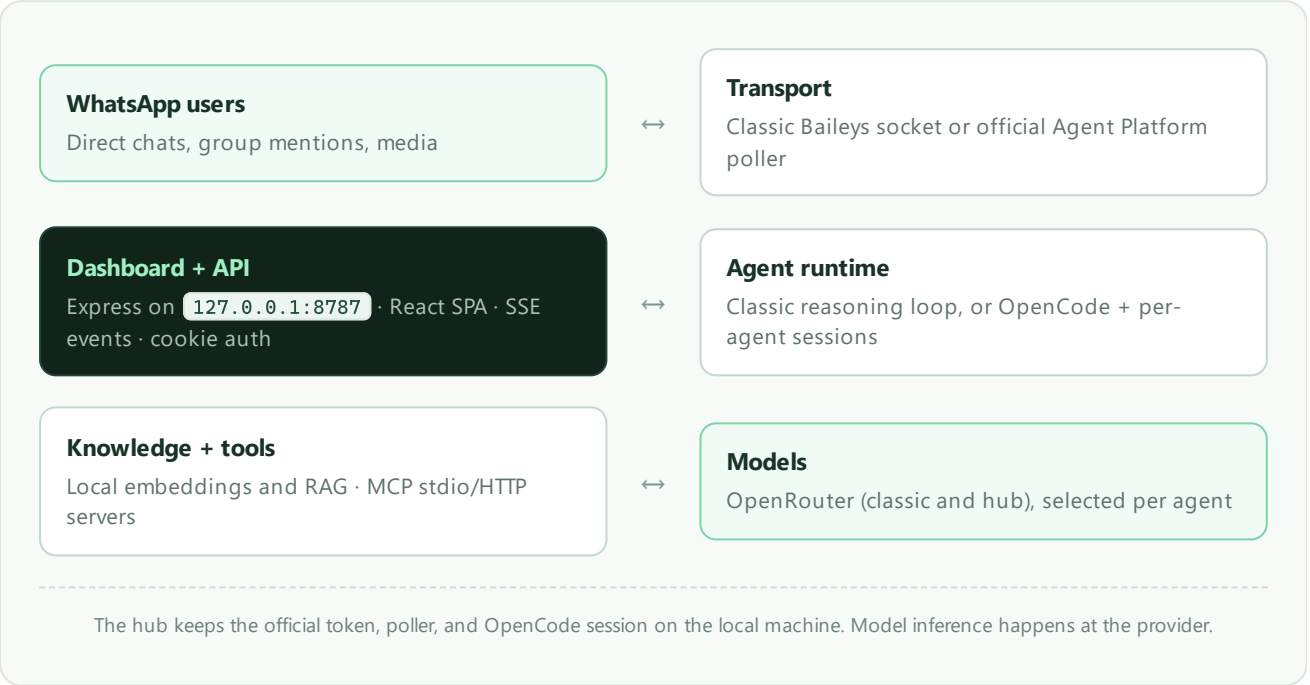
13 Security, then troubleshooting and reference

PART II · CHAPTER 6

System at a glance

Two transports, one dashboard, one SQLite database, and an OpenCode runtime for hub agents.

WhatsApp Agent is a single Node process that serves the React dashboard, runs the classic Baileys socket, and — when hub agents are active — manages a child `opencode serve` process. Everything durable lives in `data/agent.sqlite` or the data directory.



The moving parts

COMPONENT	RESPONSIBILITY
Express API	Serves the dashboard, exposes `/api/*`, handles authentication, SSE, uploads, and graceful shutdown.
SQLite	Stores settings, chats, messages, knowledge, MCP servers, agent runs, hub agents, tokens, skills, schedules, and sessions. WAL mode with a busy timeout.
Baileys	The classic WhatsApp Web socket. QR linking, session restore, reconnects, and message handling.
Official client	Long-polls the WhatsApp Agent Platform, applies per-minute rate limits, downloads media, sends replies and receipts.
OpenCode runtime	Spawns `opencode serve` on localhost, generates `opencode.json`, writes skills, creates per-conversation sessions, and runs prompts.
MCP manager	Connects stdio, Streamable HTTP, and SSE servers; discovers tools; exposes them to the classic loop and to OpenCode.
RAG	Chunks documents, embeds them locally with `all-MiniLM-L6-v2`, and combines vector and keyword search.
Scheduler	Runs enabled cron jobs with `croner` in the configured timezone and records the result.

TUI	A separate terminal client that reads `/api/hub/status` and sends start/stop/restart commands.

A message's journey through the hub

1. The poller asks the official API for new messages using a stored 64-bit offset.
2. The hub checks the message id against `hub_messages` to avoid duplicate replies after a restart or replay.
3. It records the inbound message in a conversation keyed by agent and sender, updates the agent's last-seen time, and sets the creator on the first inbound message.
4. Read receipt (and optional typing) is sent. Media is downloaded, checked, and written to `data/uploads`.
5. Attached knowledge is searched; relevant passages are prepended to the prompt.
6. OpenCode runs the prompt in the conversation's own session, with the agent's model, prompt, skills, MCP access, and file attachments.
7. The reply is sent back, stored, and recorded as an agent run with tokens and latency.
8. If the model fails, a short fallback reply is sent unless sending also fails; the error is logged.

Key properties

Local control

The dashboard, database, tokens, sessions, scheduler, and OpenCode process are local. The only outbound calls are the model provider, the official WhatsApp API, and any MCP servers you configure.

One poller per token

The official API allows one active poller per token. A second app instance receives HTTP 409. Run one copy of the app against a given token.

Per-conversation sessions

Sessions are keyed by agent and sender. Test prompts and schedules use separate keys so they do not contaminate customer conversations.

Generated configuration

`opencode.json` is derived from the hub database. Do not hand-edit it while the hub is managing the file; use the dashboard and let it regenerate.

Ports and processes

ENDPOINT	DEFAULT	NOTES
Dashboard / API	`127.0.0.1:8787`	Set `PORT` and `HOST`. Keep it on localhost unless you add a reverse proxy.
OpenCode	`127.0.0.1:4096`	Managed child process with random Basic auth. Tries the next ports if 4096 is busy.
Vite dev server	`127.0.0.1:5173`	Development only; proxies `/api` to 8787.
Official API	`api.whatsapp.com/agent/v1`	Override with `WHATSAPP_AGENT_API_BASE` only for a compatible test service.

PART II · CHAPTER 7

Install and first run

From an empty folder to a signed-in dashboard, with a safe first configuration.

The classic bot needs Node.js and an OpenRouter key. The hub additionally needs OpenCode on the PATH and a token from the official WhatsApp Agent Platform. Install and verify the base system first.

Prerequisites

- Node.js 22 or newer (tested with Node 24).
- An OpenRouter API key, or another OpenAI-compatible endpoint configured through the environment.
- For the classic bot: a phone with WhatsApp, ideally a secondary number.
- For the hub: OpenCode 2.x available as `opencode`, and an official per-agent token.
- A local folder you can back up, with enough space for SQLite, models, and uploads.

1. Install dependencies

```
cd "$env:USERPROFILE\Documents\Projects\WhatsApp-Agent"
npm install --no-audit --no-fund
```

2. Create the environment file

```
Copy-Item .env.example .env
notepad .env
```

At minimum, set a long `ADMIN_PASSWORD`, a random `APP_SECRET` of 24+ characters, and your OpenRouter key. Store `APP_SECRET` somewhere safe: it decrypts stored keys and tokens.

VARIABLE	EXAMPLE	WHY IT MATTERS
`ADMIN_PASSWORD`	a long random passphrase	Dashboard login. Without it the dashboard is inaccessible.
`APP_SECRET`	24+ random characters	Encrypts the OpenRouter key and hub tokens at rest.
`OPENROUTER_API_KEY`	`sk-or-v1-...`	Passed to the classic loop and to the OpenCode child process.
`OPENROUTER_MODEL`	`openai/gpt-4.1-mini`	Default classic model; new hub agents inherit it with an `openrouter/` prefix.
`PORT` / `HOST`	`8787` / `127.0.0.1`	Where the dashboard listens. Keep localhost for a single-machine setup.
`OPENCODE_BIN`	`opencode`	Full path when the executable is not on the PATH.
`OPENCODE_PORT`	`4096`	Base port for the managed OpenCode server.

3. Build and start

```
npm run build
npm start
```

For development, `npm run dev` starts the API and Vite together. The production build serves the compiled dashboard from `public/admin` on port 8787.

4. Sign in

Open `http://127.0.0.1:8787` and enter `ADMIN_PASSWORD`. The session cookie is HTTP-only and lasts seven days on that browser.

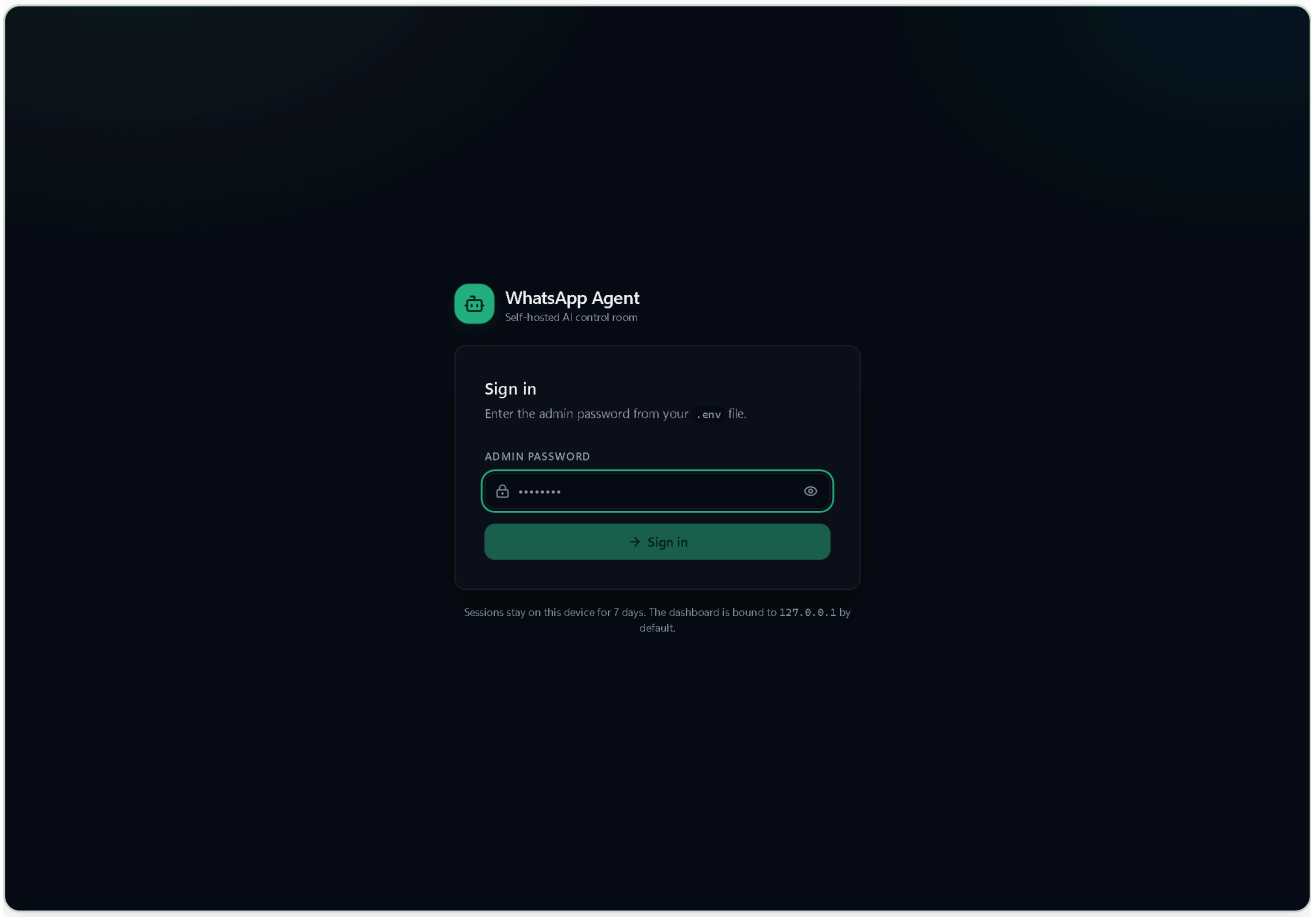


Figure 1 The dashboard sign-in. Use the password from `ADMIN_PASSWORD`; the session is local to this browser.

5. Verify the base system

- ☑ **Overview** loads and shows the classic agent's model and status.
- ☑ **Settings** accepts your OpenRouter key and reports it as configured.
- ☑ **Playground** returns a reply from the classic model.
- ☑ **Logs & runs** shows the startup sequence without repeated fatal errors.

6. Connect a transport

CLASSIC

Link with a QR code

Open **WhatsApp** → **Connect**, then scan with the phone's Linked devices screen. Keep the phone online long enough for the session to restore. Do not log out the linked device from the phone unless you intend to disconnect the bot.

HUB

Add an official token

Open **Agents** → **New agent**, paste the per-agent token, choose a model, and press **Start**. Use the test prompt before sending real traffic. Details are in chapter 10.

BEFORE YOU INVITE USERS

Confirm that `APP_SECRET` is set *before* saving keys or tokens. Changing it later makes existing encrypted secrets unreadable. Back up `.env` and `data/` together.

7. First-run checks

SYMPTOM	LIKELY CAUSE AND FIX
`EADDRINUSE` **on 8787**	Another app or an older copy is running. Stop it, or set `PORT` to a free port and use the same value in `APP_URL`.
Dashboard shows a blank page	The web build is missing. Run `npm run build` and start with `npm start`; in development, use `npm run dev`.
"Authentication required"	No dashboard session, or a cookie from another host. Sign in at `127.0.0.1` and keep that hostname.
QR code never appears	The classic socket has not started or the old session is stuck. Open **WhatsApp** and press connect; read the terminal error.
Model call returns 401	The OpenRouter key is missing, expired, or encrypted with a different `APP_SECRET`. Re-enter it and restart.
Hub start fails immediately	No token, a disabled agent, or OpenCode is not on the PATH. Fix the first error in the dashboard, then test with a free model.

PART II · CHAPTER 8

Dashboard tour

The pages you will use, what they are for, and the status badge that confuses everyone.

The left sidebar is the map. The top-right connection badge belongs to the *classic* WhatsApp session only — it is not your dashboard login status. You can be signed in successfully and still see "Logged out" because the QR-linked phone session has expired.

PAGE	USE IT FOR
Overview	The classic agent's connection, model, tool count, knowledge count, recent runs, and setup checklist.
Agents	Official hub agents: tokens, models, prompts, MCP bindings, knowledge, skills, schedules, test prompt, test send.
WhatsApp	QR linking, session state, reconnect, and logout for the classic bot.
Conversations	Stored chats and transcripts; reset context or delete a chat.
Playground	Talk to the classic agent without sending a WhatsApp message; inspect latency, tokens, and retrieval.
Knowledge	Add text or upload documents, index them, search, reindex, or delete.
MCP Tools	Create, test, enable, and edit MCP servers; inspect discovered tools.
Settings	Persona, model, generation limits, retrieval, group mode, timezone, and OpenRouter key.
Logs & runs	Live application logs and a filterable history of model calls with tokens and latency.

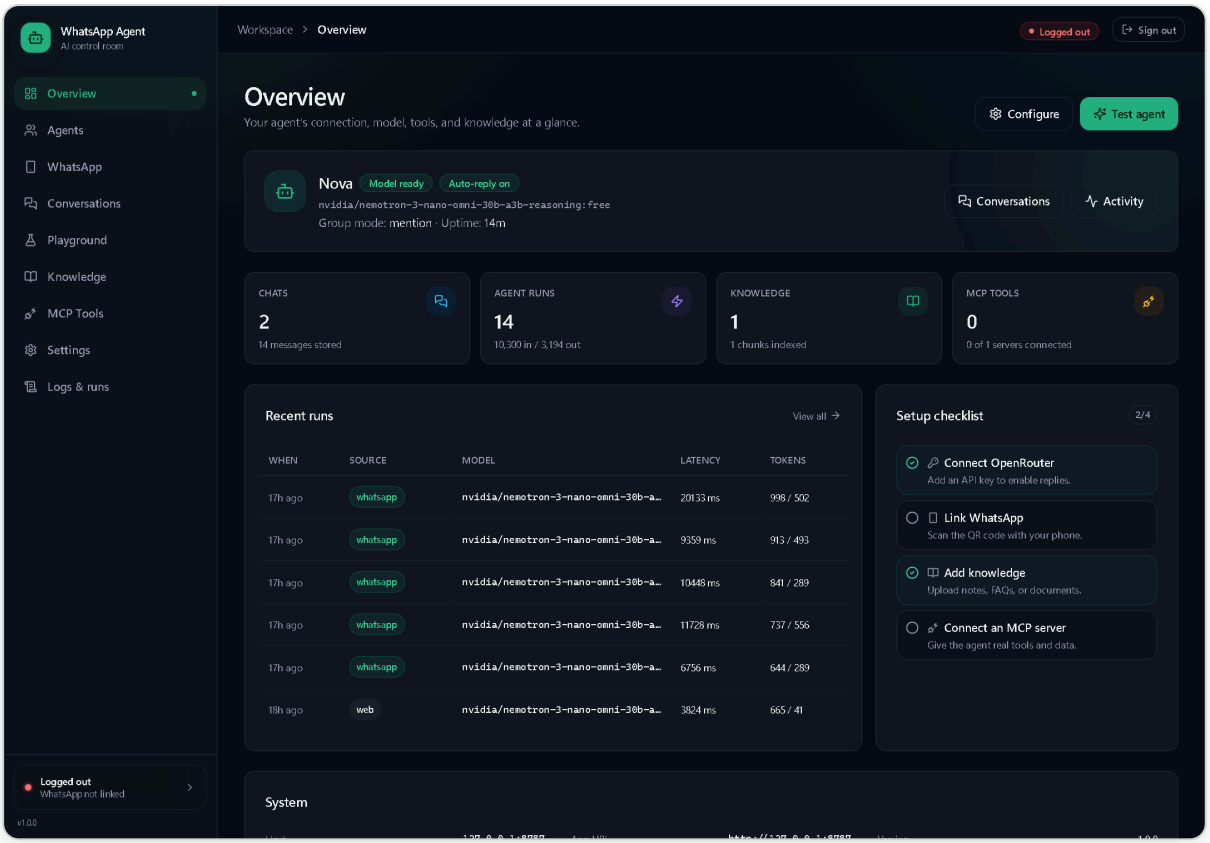


Figure 2 The Overview page. The top-left card is the classic agent; hub agents live on their own page. The statistics combine both transports where relevant.

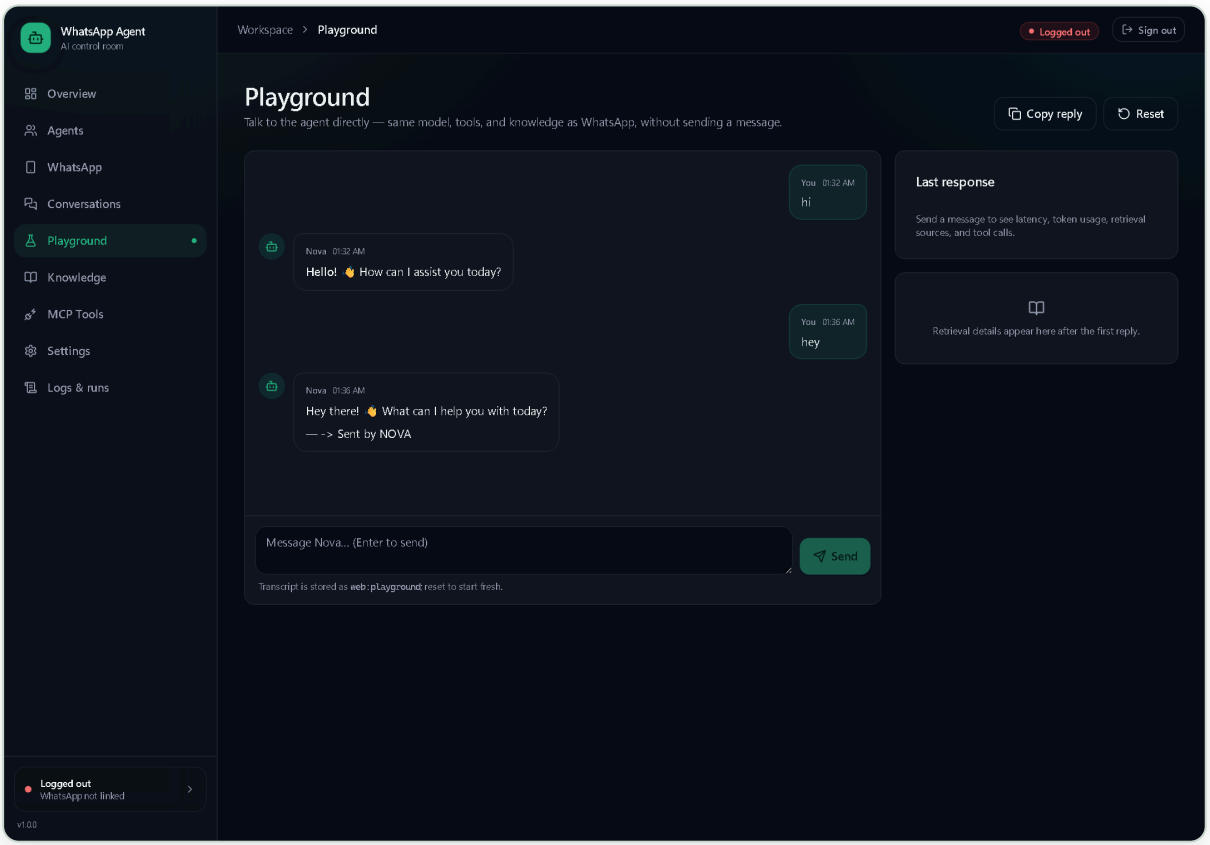


Figure 3 The Playground uses the same model, tools, and knowledge as WhatsApp without sending a message. It is the fastest way to test a prompt change.

Reading the status signals

SIGNAL	MEANING
Logged out (top right)	The classic Baileys session is not linked. Hub agents are unaffected.
Model ready	An OpenRouter key is configured and the selected model can be used.
OpenCode ready	The managed OpenCode server is running and answering on its local port.
0 of 2 agents polling	Two hub agents exist, none has an active official-token poller.
Token needed / token saved	Whether a hub agent has a stored official token. Tokens are never displayed again.
Stopped / Starting / Running / Error	The poller state for one hub agent. An error usually includes a short message.
Scheduler error	A cron expression could not be scheduled. Fix the expression or timezone.

SCREENSHOT NOTE

The sample dashboard shows a hub with two agents and a connected filesystem MCP server. Your own dashboard may show different counts, an empty state, or a red classic-WhatsApp badge.

PART II · CHAPTER 9

Classic bot administration

The QR-linked Baileys bot: persona, model, RAG, MCP, conversations, and the risks of an unofficial session.

The classic bot is the original single-agent mode. It is useful when you want a normal WhatsApp number linked by QR and you do not need per-agent official tokens. It is also the mode with the most account risk, because Baileys is not an official WhatsApp integration.

Link and maintain the session

1. Open **WhatsApp** and press **Connect WhatsApp**.
2. Scan the QR code from WhatsApp → **Linked devices** → **Link a device**.
3. Wait for the state to become **connected** and the phone number to appear.
4. Keep `data/wa-auth` in your backups. It contains the session credentials.

A logged-out phone or a removed linked device forces a new QR scan. The dashboard shows the last error. If the session is repeatedly closed with status 401, remove the old session and link again rather than restarting the server in a loop.

Persona and generation

Settings → **Persona** sets the bot name, timezone, and system prompt. The timezone is also used by hub schedules. **Model & generation** sets the OpenRouter model, temperature, maximum tokens, history turns, tool steps, and tool timeout. Lower history and tool steps reduce cost and latency; higher values help complex follow-ups.

Retrieval settings

SETTING	EFFECT
Enable retrieval	Turns knowledge search on or off for the classic loop.
Auto-inject context	Adds the best passages to the prompt automatically; turn off to require explicit searches instead.
Top K	How many passages to retrieve. Higher is not always better; too many dilute the answer.
Min score	The relevance floor. Raise it to avoid weak matches; lower it if useful material is being skipped.

Group behaviour

Group mode controls whether the bot ignores groups, replies only when mentioned, or replies to every message. Mention-only is the safest default. Group messages are more likely to contain unrelated chatter and prompt-injection attempts.

Knowledge and RAG

1. Open **Knowledge** and add text or upload a supported file (TXT, MD, CSV, JSON, HTML, PDF, DOCX).
2. Wait for indexing. Each document is chunked and embedded locally; the first run downloads the embedding model.
3. Use **Search knowledge** to confirm that a realistic question retrieves the right passage.
4. Reindex after changing the embedding model or editing a source file. Delete obsolete documents rather than leaving contradictions.

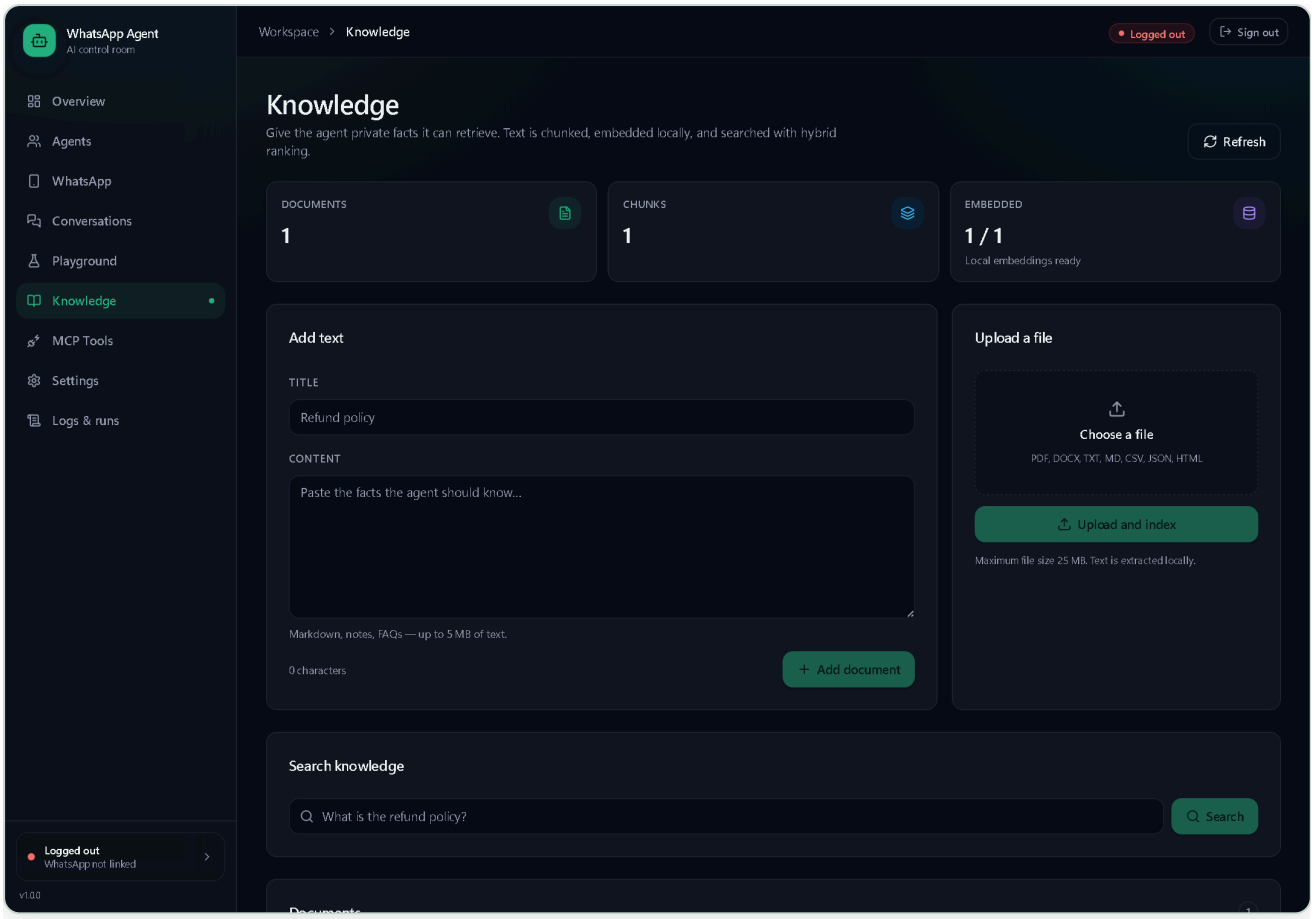


Figure 4 Knowledge is shared locally. Hub agents can attach a subset of these documents; the classic bot uses retrieval settings from the dashboard.

MCP tools

Add a server on **MCP Tools**, test the connection, then enable it. The classic loop sees the tool names exposed by every enabled server. Disable tools you do not want the model to call. A stdio MCP server runs as a child process with your user privileges; treat its command line as code you trust.

> **UNOFFICIAL TRANSPORT**
>
> Baileys can break when WhatsApp changes its web protocol, and automation can put the linked number at risk. Use a secondary number, do not spam, and keep an official-token hub agent as the migration path for production traffic.

Everyday checks

- [x] The Overview badge is green and the phone number is correct.
- [x] The Playground answers with the current model and prompt.
- [x] Recent runs show reasonable latency and no repeated tool errors.
- [x] Knowledge search finds the policy you expect.
- [x] MCP servers that should be connected show a tool count.

PART II · CHAPTER 10

Multi-agent hub

Several official WhatsApp numbers, each with its own brief, model, tools, knowledge, skills, and schedule — all on one local runtime.

The hub is a different transport and a different runtime from the classic bot. Each hub agent owns an official token, a poller, an OpenCode configuration entry, and per-conversation sessions. The dashboard is the control plane; OpenCode does the reasoning and tool work.

> **THE MENTAL MODEL**
>
> An **agent** is a row in SQLite plus a generated block in `opencode.json`. A **poller** is the process loop that reads one token. A **session** is the model's memory for one sender. None of these are shared between agents unless you deliberately share a knowledge document or an MCP server.

10.1 Prerequisites and the official token

- [x] OpenCode 2.x is installed and `opencode --version` works from the same shell that starts the app.
- [x] The app's OpenRouter key is configured; it is passed to the OpenCode child process as `OPENROUTER_API_KEY`.
- [x] You have a token for the specific WhatsApp number from the official WhatsApp Agent Platform.
- [x] The token has the permissions required to read messages, send messages, and update read/typing status for that number.

Treat the token like a password. It is encrypted with `APP_SECRET`, never shown again in the dashboard, and never written into `opencode.json`. One token belongs to one number and one active poller. Do not paste the same token into two agents or run two copies of the app against it.

10.2 Create and configure an agent

1. Open **Agents** and press **New agent**.
2. Give it a name that tells you the number's job, not just the person: “Customer Support”, “Order Updates”, “Events Desk”.
3. Enter a model in OpenCode form, for example `openrouter/openai/gpt-4.1-mini`.
4. Paste the official token, write the system prompt, and save.
5. Configure MCP, knowledge, skills, and schedules while the agent is stopped. Save each panel.
6. Use **Test prompt** to confirm the model and prompt work, then **Start** the poller.
7. Send a real WhatsApp message from the creator's phone and confirm the reply before handing the number to users.

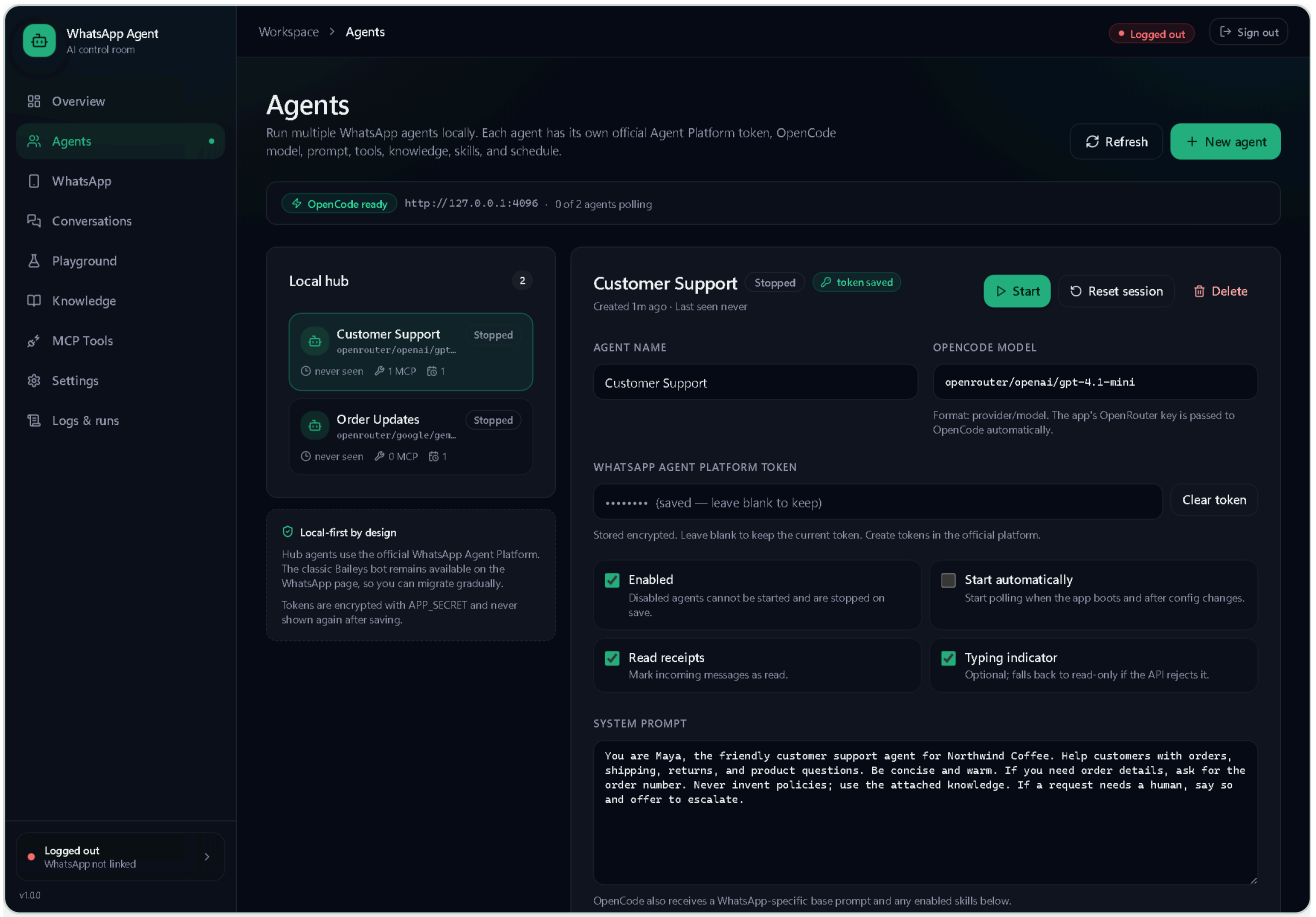


Figure 5 The hub workspace. The left column lists agents and their lifecycle state; the right column edits the selected agent. "Token saved" means the encrypted token exists, not that it was validated.

10.3 Model and system prompt

Hub models use OpenCode's provider-prefixed form. For OpenRouter models the prefix is `openrouter/`, followed by the provider and model id. The model must be one your OpenRouter key can access and afford; a free model is useful for smoke tests but may have rate limits or weaker tool use.

FIELD	GUIDANCE
Model	Use `openrouter/<provider>/<model>`. New agents inherit the classic model with the prefix added.
System prompt	The agent's brief: role, audience, tone, facts it must not invent, escalation rules, and boundaries. The runtime prepends WhatsApp formatting and safety instructions automatically.
Enabled	Disabled agents cannot be started; saving the disabled state stops a running poller. Their settings, skills, and schedules remain stored.
Auto-start	Starts the poller when the app boots. Use it for production agents; leave it off while experimenting.
Read receipts	Marks inbound messages as read through the official API.
Typing indicator	Optional. If the API rejects it, the app falls back to the read receipt and continues.

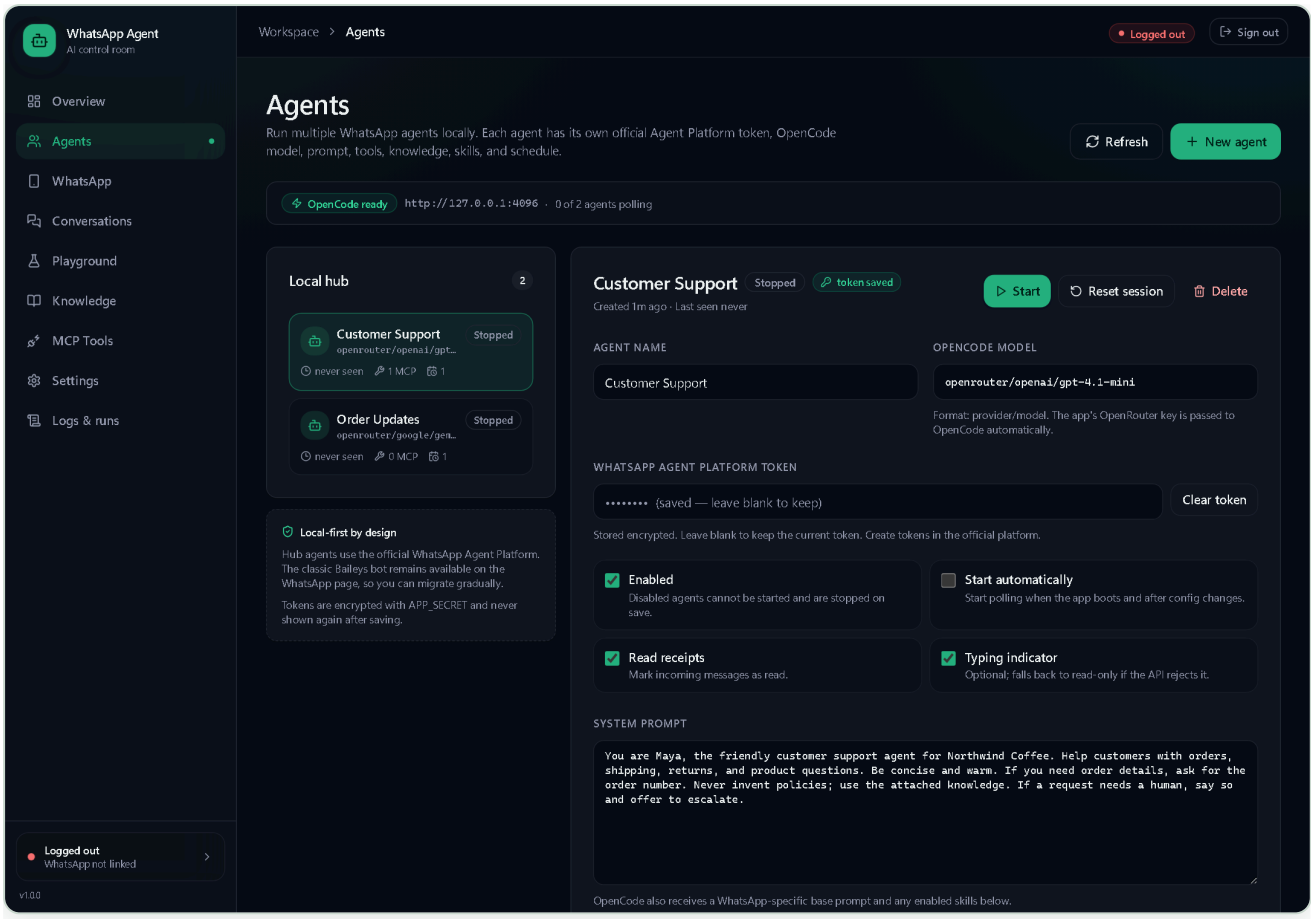


Figure 6 Configuration stays in one panel: identity, model, prompt, official token, and lifecycle flags. The token field is write-only.

> **MODEL QUOTAS ARE REAL**
>
> OpenCode reports provider errors, including insufficient credits and context-length failures, in the run result and the dashboard log. A hub agent that answers "I could not handle that message" may have a model quota problem rather than a WhatsApp problem. Check **Logs & runs** and the runtime log tail before changing the token.

10.4 MCP access, isolated per agent

MCP servers are created once on **MCP Tools**, then bound to individual agents. The hub writes every enabled server into `opencode.json`, but adds explicit deny rules for every server an agent is *not* bound to. Tool names are exposed with a normalized server prefix, so a server named `filesystem` exposes tools such as `filesystem_read_file`. The runtime also disables OpenCode's generic MCP resource listing and reading for unbound servers.

- **Bind only what the agent needs.** A support agent does not need a deployment tool.
- **Review disabled tools.** The MCP server can hide individual tools from the model without disconnecting the server.
- **Prefer read-only tools** for customer-facing agents. Any write tool can be triggered by a persuasive message or a malicious document.
- **Test before starting.** Use the MCP **Test** button to confirm the command or URL, credentials, and tool list.

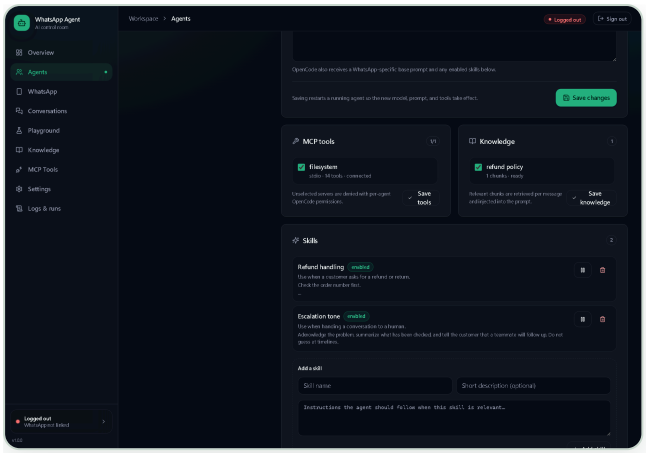


Figure 7 Bindings are visible per agent. The badge shows bound servers over enabled servers; each row shows connection state and tool count.

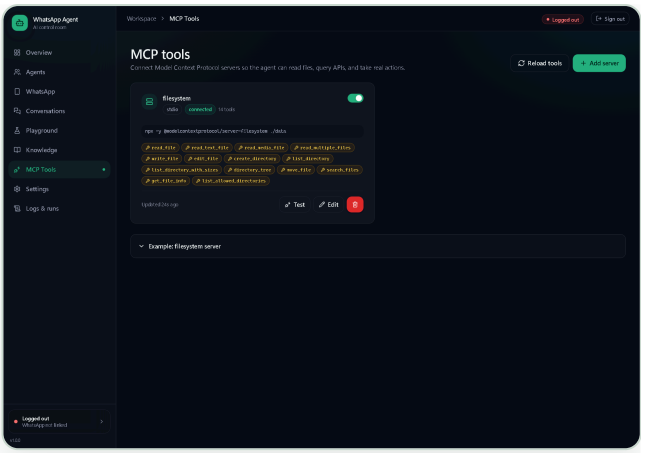


Figure 8 MCP servers are managed centrally, then enabled, tested, and bound. A stdio server runs as a local child process.

10.5 Knowledge

Knowledge is uploaded once and can be attached to several agents. The hub searches only the documents attached to the current agent, and injects the best passages before the prompt reaches OpenCode. This is useful for policies, product facts, price lists, and internal procedures.

1. Upload and index the document on **Knowledge**.
2. Search it with a realistic question and check the retrieved passage.
3. Attach it to the agent and save.
4. Ask the agent a question that requires the document; verify that it cites or follows the content.

10.6 Skills

A skill is a reusable block of Markdown instructions. Skills are stored in SQLite, written to `.opencode/skills/wa-hub-*` on each config sync, and appended to the agent's system prompt. Use them for procedures that would otherwise make every agent's prompt huge: refund handling, escalation tone, status vocabulary, or a checklist for a specific task.

Good skill

Short, scoped, and testable. “When a customer asks for a refund: confirm the order, apply the 30-day window, offer a replacement first, escalate above 100 USD.”

Bad skill

A second copy of the whole company handbook, contradictory policies, or instructions that grant new tool permissions. Skills shape behaviour; they do not grant access.

10.7 Schedules

Schedules run cron prompts on the local scheduler. The reply is sent to the agent's **creator** — the first WhatsApp sender recorded for that agent — and stored in that creator's conversation with a separate `schedule` model context. Schedules run while the app is running even if the agent's poller is stopped, provided the agent still has a token and a creator.

RULE	DETAIL
Cron syntax	Validated when saved. Standard five-field cron is enough for most jobs: minute, hour, day of month, month, day of week.
Timezone	Uses the timezone in **Settings**. An empty timezone falls back to the server's local timezone.

Creator required	If no one has messaged the agent, the job records an error instead of sending. Send the agent a message first.
Manual run	The **Run now** button executes the same path immediately; use it to test the prompt and delivery.
Failure handling	A failed model call or send is recorded on the schedule and in the logs. It does not retry forever.

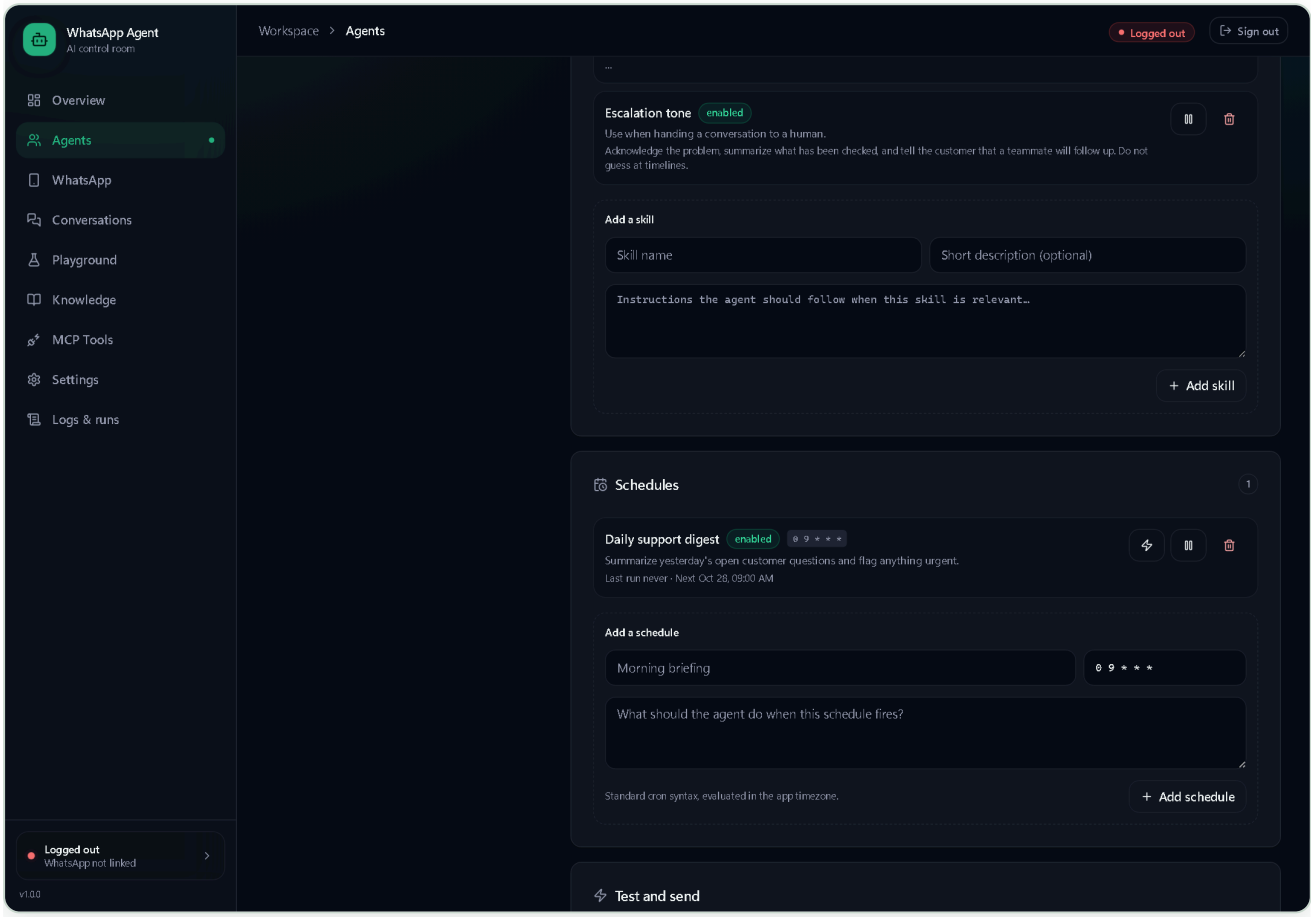


Figure 9 Schedules belong to an agent. The example uses a daily support digest and a weekday order summary; the output goes to the creator's WhatsApp chat.

10.8 Start, stop, restart, and reset

- **Start** creates the official client, verifies that a token exists, ensures OpenCode is running, syncs the generated config, and begins polling.
- **Stop** aborts the poller but keeps the token, sessions, skills, and schedules. Schedules can still run.
- **Reset session** stops the agent, clears every stored OpenCode session for that agent, and restarts it if it was running. The dashboard transcripts remain.
- **Delete** stops the agent, removes its token, bindings, skills, schedules, sessions, and generated config entry. This cannot be undone.

> **HTTP 409 IS A TOKEN COLLISION**
>
> The official API allows one active poller per token. If a second app instance or a leftover process polls the same token, the API returns HTTP 409. Stop the duplicate process; do not create a second agent with the same token.

10.9 Conversation and media behaviour

Each sender gets a separate OpenCode session and a separate dashboard conversation named after the agent and sender. Test prompts use a `test` context, schedules use a `schedule` context, and customer chats use the sender's id. This prevents a test or a scheduled job from inheriting a customer's conversation — and prevents customers from inheriting each other's.

Inbound images, audio, video, and documents are downloaded through the official API and written to `data/uploads`. Files larger than 20 MB are skipped to protect the local runtime. The file is attached to the OpenCode prompt; whether the model can interpret it depends on the selected model's modality support. If an attachment is ignored, ask the user to send the text or convert the file.

10.10 Generated OpenCode configuration

The runtime generates `opencode.json` at the project root from the current agents, enabled MCP servers, bound knowledge, and skills. It contains:

- A default model and one `agents.wa_hub_*` block per hub agent.
- The full system prompt, including base WhatsApp instructions and enabled skills.
- MCP server definitions, with `codemode: false`, and deny permissions for unbound servers.
- No WhatsApp tokens, no OpenRouter key, and no dashboard password.

If an existing unmanaged `opencode.json` is present, it is backed up once as `opencode.json.bak-<timestamp>`. `data/opencode-managed.json` stores the hash of the file the app wrote, so it is not backed up repeatedly. Saving an agent, deleting an agent, or changing an MCP server refreshes the file. The running OpenCode server reloads it when it is already up; otherwise the file is simply written for the next start.

> **DO NOT HAND-EDIT WHILE THE HUB RUNS**
>
> The next save will overwrite your edits. If you need to inspect or debug the generated file, copy it aside first. To force a clean regeneration, use the runtime reload control or restart the app after making the change in the dashboard.

10.11 Hub checklist

- [ ] `opencode --version` succeeds in the app's environment.
- [ ] The agent has a token and the status is **Running**, not just **Enabled**.
- [ ] The creator is recorded and can receive a test send.
- [ ] Only the intended MCP servers are bound; write tools are reviewed.
- [ ] Knowledge is attached and a realistic question retrieves the right passage.
- [ ] Skills are short, non-contradictory, and tested.
- [ ] Schedules have a creator, a valid timezone, and a successful manual run.
- [ ] The generated config contains the expected agents and no deleted MCP servers.

PART II · CHAPTER 11

Terminal monitor

A small TUI for the moments when opening a browser is more effort than it is worth.

`npm run tui` signs in to the local API, shows the classic WhatsApp state, the OpenCode runtime, and every hub agent, and lets you start, stop, or restart an agent without leaving the shell.

```
cd "$env:USERPROFILE\\Documents\\Projects\\WhatsApp-Agent"
npm run tui
```

The TUI reads `ADMIN_PASSWORD` from `.env` when it is present; otherwise it asks for the password with hidden input. It connects to `APP_URL` if set, or to `http://127.0.0.1:8787` by default. The screen refreshes every five seconds.

```
WhatsApp Agent Hub  - http://127.0.0.1:8787
classic WhatsApp: connected (+1 555 010 7788)   OpenCode: ready   polling: 2/2
------------------------------------------------------------------------------------------

  > Customer Support         running   gpt-4.1-mini        creator: set
    Order Updates            running   gemini-2.5-flash    creator: set

selected agent  Customer Support
status          running        auto-start: yes   enabled: yes
mcp servers     filesystem (14 tools)   knowledge: 1 doc
schedules       1 enabled      last seen: 2 minutes ago

up/down select  -  s start/stop  -  r reset + restart  -  R refresh  -  q quit
Illustrative view; agent names and counts depend on your hub.
```

Keys

KEY	ACTION
`↑` / `k`	Select the previous agent.
`↓` / `j`	Select the next agent.
`s`	Start the selected agent, or stop it if it is already running.
`r`	Reset the agent's session and restart it. This clears model context for every conversation.
`R`	Refresh the screen immediately. The automatic refresh continues every five seconds.
`q` / `Ctrl+C`	Quit. The API server keeps running.

WHAT THE TUI CAN AND CANNOT DO

It can start, stop, and reset agents, but it is not a full editor. Creating agents, binding MCP servers, editing prompts, and managing schedules still happen in the dashboard. The TUI is a monitor and a fast lifecycle switch.

When the TUI will not connect

- Confirm the API is running and `APP_URL` points to the right host and port.

- If `ADMIN_PASSWORD` is not in the environment used to launch the TUI, it prompts. A wrong password hits the normal login rate limit.
- The TUI shows the API error text at the bottom of the screen; fix the API first, then press `R`.

Reading an agent line

FIELD	WHAT TO CHECK
running / stopped / error	The poller state. **Running** means the official poller is active, not merely that the profile is enabled.
Model	The final component of the OpenCode model id. Confirm it is the model you intended, especially after a quota change.
creator: set / missing	Whether the first sender is recorded. Schedules and test sends fail without a creator.
MCP servers	Bound server names and tool counts. A server that was enabled in the dashboard but not bound should not appear here.
Knowledge / schedules	Attached document count and enabled cron job count. Zero is valid for a new agent.
Last seen	The last inbound message time. A running agent with no traffic is normal; a growing gap during business hours may not be.

A five-minute morning check

1. Confirm the classic badge matches the session you expect. A hub-only deployment can stay logged out.
2. Check that every agent you expect to serve traffic is **running**, not just enabled.
3. Open the agent with the highest traffic and confirm its model, creator, and MCP counts.
4. Press `R` once and look for an error line. The automatic refresh continues every five seconds.
5. Open **Logs & runs** in the dashboard for quota errors, repeated tool failures, or a fallback storm.

> **STATUS IS NOT HEALTH**
>
> A green **running** label only proves that the poller has not stopped. It does not prove that the model has credit, that the token still has the right scopes, or that replies are reaching users. Send one real test message after any token, model, or prompt change.

Operations, limits, and backups

The day-two work: capacity, token collisions, model cost, upgrades, and a backup you can actually restore.

A working demo is not an operating service. The hub adds external rate limits, a child runtime, and long-lived tokens to the classic bot's usual concerns. This chapter is the maintenance checklist.

Official API limits

OPERATION	DOCUMENTED CAP	APPLIED IN THIS APP
Message sends	12 / minute	11 / minute, with a send queue
Status updates (read/typing)	12 / minute	11 / minute
Polls	15 / minute	14 / minute
Media operations	12 / minute	11 / minute

The internal limiters keep a small margin under the published caps. They are per agent. Sending a burst of WhatsApp replies may queue rather than fail; a sustained burst is still a design problem. Split long messages deliberately and avoid sending duplicate answers.

One poller per token

The official API tracks the active poller for each token. Running the app twice against the same token returns HTTP 409 and can leave both instances confused. Run one API process per token. If you need two numbers, create two agents with two tokens; they can share one app process and one OpenCode server.

Poll offsets are stored as 64-bit values in SQLite. A replayed offset is deduplicated through `hub_messages`, so a restart does not normally produce duplicate replies. If you deliberately reset an offset, expect the API to replay messages; the dedupe table is the safety net.

OpenCode lifecycle and ports

- OpenCode starts on the first hub prompt or agent start, not when the dashboard is opened.
- It listens on `127.0.0.1` with a random Basic-auth password. The app starts at `OPENCODE_PORT` (4096) and tries a small number of following ports if it is busy.
- The process stays up while the app runs, even if every poller is stopped. It stops during graceful app shutdown.
- If OpenCode crashes, the next prompt or start calls `ensure()` and respawns it. The configuration is written again and reloaded.
- Do not expose the OpenCode port. It has its own authentication, but it is not designed as a public service.

Schedules and polling are independent

A stopped poller does not disable schedules. If the agent still has a token and a recorded creator, enabled cron jobs continue to run and send results while the app is up. This is useful for morning digests, but it can surprise you during maintenance. Pause the schedules or stop the app if you need a complete quiet period.

Model cost and capacity

PRESSURE	WHAT HAPPENS	RESPONSE

Credits exhausted	OpenCode returns a provider error; the hub sends the fallback reply.	Top up OpenRouter, switch to a cheaper model, or use a free model for tests.
Context too long	The model rejects the prompt or truncates history.	Reset the session, shorten the prompt, reduce attachments, or summarize earlier turns.
Slow tools	Replies arrive late or the run fails on timeout.	Disable slow MCP tools for that agent; increase the tool timeout only after measuring.
Rate limits	The provider returns 429 or a queue delay.	Lower concurrency, add a cheaper fallback model for bursts, or reduce schedule frequency.

Backups

The simplest reliable backup is a stopped-app copy. SQLite in WAL mode is safe to copy while the app is running only if you use a SQLite-aware backup tool; a raw file copy can miss the WAL. For a single-machine deployment, stop the app, copy the data, then start it again.

```
# Stop the app first (Ctrl+C in the terminal running npm start)
Copy-Item .env "D:\\Backups\\whatsapp-agent\\env-$(Get-Date -Format yyyyMMdd).bak"
Copy-Item data "D:\\Backups\\whatsapp-agent\\data-$(Get-Date -Format yyyyMMdd)" -Recurse
```

What to keep together:

- `.env` — especially `APP_SECRET`; without it the encrypted secrets are unreadable.
- `data/agent.sqlite` and its WAL/SHM files if copied live.
- `data/wa-auth` — the classic session credentials.
- `data/uploads` — inbound hub media.
- `opencode.json` and `data/opencode-managed.json` — optional, because they can be regenerated, but useful for debugging a specific configuration.

`data/models` is a cache; you can omit it and let the app download the embedding model again. Backups contain plaintext messages, prompts, and media. Encrypt the backup destination and restrict access.

Restore and upgrade

1. Stop the app. Restore `.env` and `data/` from the same backup set.
2. Run `npm install` and `npm run build` for the target version.
3. Start the app and sign in. Schema migrations run automatically on startup.
4. Check the classic session, hub agent states, the generated config, and one test prompt.
5. Reconnect WhatsApp or re-enter tokens only if the restored secrets cannot be decrypted.

Upgrades may add tables or columns. Back up before every upgrade, and read the release notes for changes to the generated OpenCode configuration, official API behaviour, or model defaults.

Disk and hygiene

- The embedding model cache is roughly tens of megabytes; uploads grow with customer traffic.
- Review `data/uploads` on a schedule. Deleting a chat does not delete its media files.
- The application log buffer is in memory and bounded; agent runs and messages are in SQLite and grow until deleted.
- Keep at least a few gigabytes free for SQLite, media, logs, and OpenCode.
- Do not store unrelated files in the project root; the generated config and backup files live there.

Health checks

- [x] `http://127.0.0.1:8787` loads and the dashboard session works.
- [x] The classic badge matches reality; a hub-only deployment can stay logged out.
- [x] **Agents** shows the expected number of pollers and no repeating errors.
- [x] **Logs & runs** shows recent model calls with sane latency and no quota storm.
- [x] `opencode.json` contains only current agents and MCP servers.
- [x] A backup from the last day exists and its `APP_SECRET` matches the data.

Reading the logs

The log view is the first place to look after a status change. Filter by the agent name, then read the earliest error in the run rather than the final fallback message.

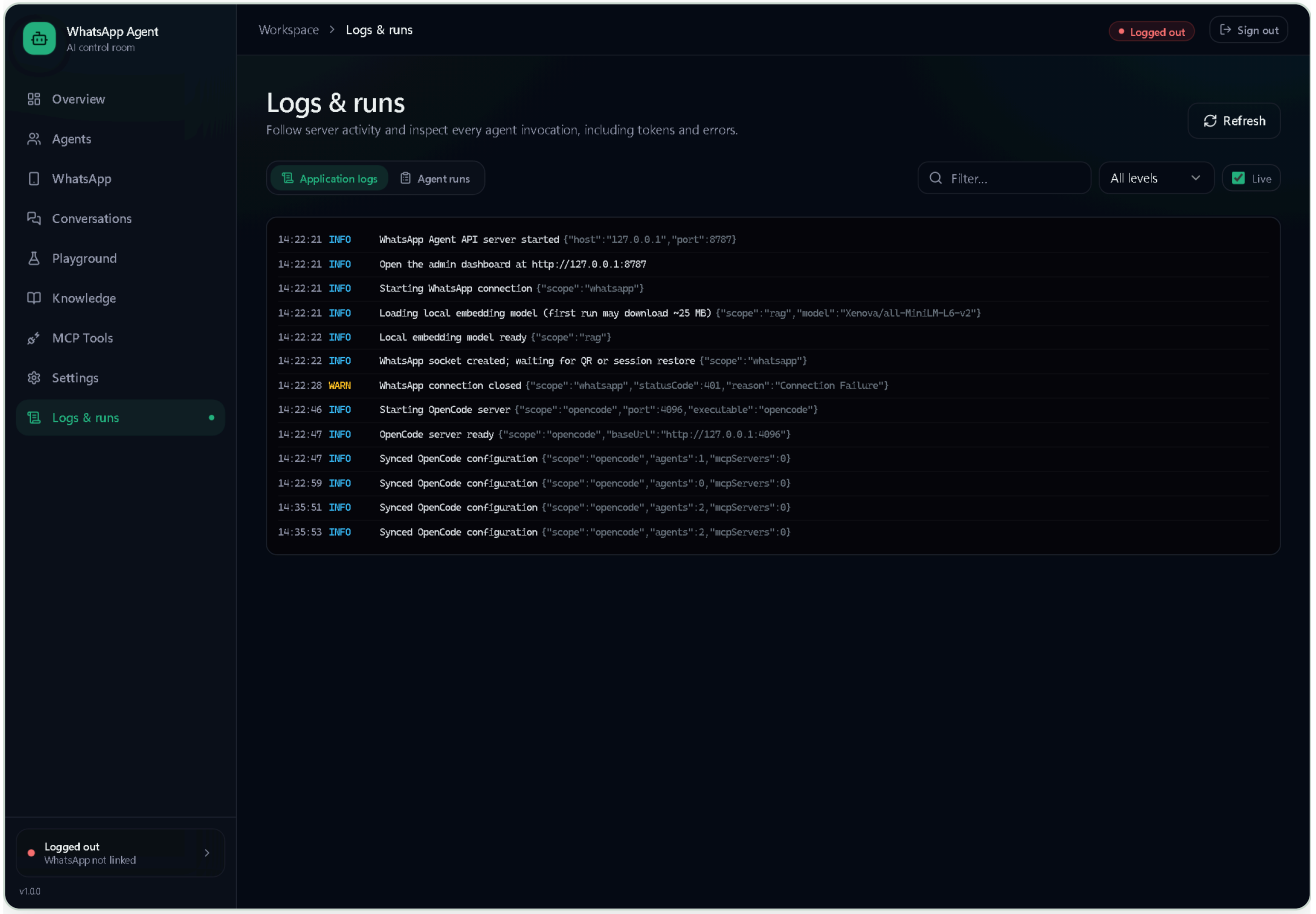


Figure 10 Logs & runs combines live application events with a searchable history of model calls, tokens, latency, and tool failures.

- **Quota or 429:** the provider rejected the call; check credit and concurrency before changing the agent.
- **Repeated tool errors:** the MCP server is down or the model is calling a disabled tool. Test the server on **MCP Tools**.
- **Fallback storm:** many failures in a row usually mean one shared cause — token, model, or runtime — not many independent user problems.

Security checklist

What to lock down before a real number, a real customer, or a real token touches the system.

The default deployment is a localhost tool for one operator. That is a reasonable starting point, not a finished security posture. Every step beyond localhost — a reverse proxy, a public number, a remote MCP server — adds a decision.

Network exposure

SURFACE	BASELINE	IF YOU EXPOSE IT
Dashboard / API	Bind `127.0.0.1`; reach it through a local browser.	Put it behind a TLS reverse proxy, restrict source IPs, and keep the admin password strong. Do not open 8787 directly.
OpenCode server	Localhost, random Basic-auth password, managed by the app.	Do not expose it. It is an internal runtime, not a public API.
Vite dev server	Localhost only, development use.	Do not run development mode on a public interface.
Official API token	Encrypted in SQLite; never displayed after save.	Grant the minimum scopes, rotate if leaked, and keep one token per number.
MCP servers	Only servers you trust; least privilege.	Treat remote MCP URLs and headers as secrets; use TLS and scoped credentials.

Authentication and secrets

- Use a long, unique `ADMIN_PASSWORD`. Login failures are rate-limited, but a weak password is still weak.
- Set a random `APP_SECRET` of 24+ characters *before* storing keys. It is the only key to the AES-256-GCM encrypted secrets.
- Changing `APP_SECRET` makes existing OpenRouter keys and WhatsApp tokens unreadable. Back up the old value or plan to re-enter the secrets.
- The dashboard cookie is HTTP-only and same-site oriented. Use the same hostname (for example always `127.0.0.1`) to avoid confusing cookie scope.
- Never commit `.env`, `data/`, backups, or screenshots containing tokens or customer data.

Generated configuration

`opencode.json` contains the system prompts, skill text, MCP server definitions, and deny rules. It does **not** contain WhatsApp tokens or the OpenRouter key. It can still contain confidential instructions, internal URLs, and MCP headers. Restrict file permissions and treat it as sensitive. The generated file is overwritten by the dashboard; do not paste secrets into it.

MCP trust

- A stdio MCP server is a program running with your user privileges. Read its command line before enabling it.
- Prefer read-only tools for customer-facing agents. A write tool is an authorization decision, not a feature toggle.

- Bind only the servers an agent needs. Unbound servers are explicitly denied, but the bound ones are fully available unless individual tools are disabled.
- Remote MCP servers receive the data the model sends. Review their privacy and retention terms.
- Test a new server in isolation before giving it to a production agent.

Prompt injection and untrusted inputs

Treat every document, web page, image, and tool output as untrusted content. An attacker can embed instructions that try to make the model ignore its brief, reveal system prompts, or call a tool. Defences are layered: keep tool permissions narrow, avoid exposing secrets to the model, separate instructions from retrieved content, and review surprising tool calls. No prompt alone is a security boundary.

Data protection

- Decide what may be stored before inviting users. Messages and media are persisted locally until deleted.
- Backups are plaintext unless you encrypt the destination. Protect them like the live database.
- Redact personal data from screenshots, support tickets, and logs.
- Delete obsolete knowledge and chats on a schedule; do not rely on automatic expiry, because there is none.
- If you handle regulated data, complete your own retention, consent, and vendor review. This guide is not legal advice.

WhatsApp transport risk

The classic bot uses Baileys, an unofficial WhatsApp Web client. It can stop working when WhatsApp changes its protocol, and automation can put the linked number at risk of restriction. Use a secondary number, avoid bulk or unsolicited messaging, and migrate production traffic to official-token hub agents. The official transport is not risk-free either: follow the platform's policies and rate limits.

Pre-launch checklist

- [ ] Dashboard bound to localhost or behind TLS with source restrictions.
- [ ] Strong admin password; login rate limiting understood.
- [ ] Random `APP_SECRET` stored with the backup set.
- [ ] Official tokens scoped and rotated on a schedule.
- [ ] MCP servers reviewed; write tools minimized; disabled tools documented.
- [ ] Prompts checked for secrets and unnecessary personal data.
- [ ] Retention and deletion policy written down and followed.
- [ ] Backups tested by restoring to a clean folder.
- [ ] One poller per token; duplicate app instances eliminated.
- [ ] Incident contact and token-revocation procedure known.

Admin troubleshooting & FAQ

Start with the layer that failed: dashboard, classic transport, hub transport, model, tools, or storage.

Most failures leave a specific message in **Logs & runs**, the hub runtime log tail, or the terminal that started the app. Read the first error, not the last symptom.

Dashboard and authentication

SYMPTOM	LIKELY CAUSE	FIX
"Authentication required" on every call	Missing or expired cookie; wrong hostname; session after restart.	Sign in again at the same host. Check the browser is not blocking the cookie. Wait out the 15-minute login lockout if needed.
"ADMIN_PASSWORD is not configured"	Empty `ADMIN_PASSWORD`.	Set it in `.env` and restart. Do not expose the dashboard without one.
Login works, then immediately signs out	Changing `APP_SECRET` or a cookie scoped to a different host.	Use one hostname; restore the original secret or sign in again after the change.
Too many login attempts	Rate limiter after repeated failures.	Wait 15 minutes. Check the password in `.env`; avoid retry loops.

Classic WhatsApp

SYMPTOM	LIKELY CAUSE	FIX
No QR code	Session already exists; socket not started; UI not refreshed.	Open **WhatsApp**, press connect, wait for the socket. If the phone is still linked, disconnect it first.
Logged out / status 401	The linked device was removed or the session expired.	Link again with a new QR code. Backups of `wa-auth` only help if the session is still valid.
Connected but no replies	Auto-reply off, group mode blocking, model error, or tool loop.	Check Settings, Playground, and Logs. Send a direct message that mentions the bot if group mode is on.
Repeated disconnects	Network instability, protocol change, or multiple sessions on the number.	Use one linked session; keep the phone online; check the terminal for the disconnect reason.

Hub agents

SYMPTOM	LIKELY CAUSE	FIX
Start button disabled	No token, or the agent is disabled.	Save a token and enable the agent. A disabled agent cannot be started by design.
HTTP 409 on start	Another poller owns the token.	Stop the duplicate app or process. Use one token per agent.
Running but no replies	Token scope, offset, official API outage, or a runtime error before send.	Check the hub log tail and the official API status. Remove and re-add the token if its permissions changed. Look for the fallback reply.

"OpenCode is not running"	Executable missing, port exhausted, or the child crashed.	Run `opencode --version` in the app's shell. Set `OPENCODE_BIN` to the full path. Check `OPENCODE_PORT` and logs.
"OpenCode rejected the generated configuration"	Invalid model id, MCP definition, or permission block.	Fix the model or MCP server, save again, and reload. Inspect `opencode.json` for the last change.
Fallback reply	Model error, quota, context length, or tool failure.	Read the run error. Switch model or top up credits; reset the session if the context is too long.
Agent deleted but config still mentions it	Stale generated file from an older build or a failed refresh.	Save any agent or MCP server to refresh, or restart the app. Confirm the managed hash file is writable.

Models and tools

SYMPTOM	LIKELY CAUSE	FIX
Quota or credit error	The OpenRouter account cannot afford the selected model.	Use a cheaper or `:free` model for tests; top up for production. Check the model id is valid.
Context length error	Long conversation, large attachment, or too much knowledge.	Reset the session, reduce top K, split the file, or use a larger-context model.
MCP server will not connect	Wrong command/URL, missing credentials, or `npx` / `node` not on PATH.	Use the MCP **Test** button. Run the command manually in the project folder. Check remote TLS and headers.
Tools missing for one hub agent	Server disabled, not bound, or the agent was not saved.	Enable the server, bind it, save the agent, and restart. Unbound servers are denied by design.
Tool call loops	The model repeats a failing call or the tool result is unclear.	Disable the tool, improve its description, reduce the step budget, or switch models.
Skills not applied	Skill disabled, agent not restarted, or the generated file was not refreshed.	Enable the skill, save the agent, restart it, and check the prompt in `opencode.json`.

Knowledge, media, and storage

SYMPTOM	LIKELY CAUSE	FIX
Embeddings unavailable	Model download failed or no network access to Hugging Face.	Check `LOG_LEVEL=debug`, network/proxy settings, and disk space. Retry the upload.
Wrong knowledge retrieved	Stale or contradictory documents; top K too high/low.	Delete obsolete documents, reindex, tune top K and min score, and test with realistic questions.
Attachment ignored	Over 20 MB, unsupported type, or a model without that modality.	Convert to text/PDF, split the file, or choose a multimodal model. Media over 20 MB is skipped.
Disk filling up	Uploads, model cache, database, or old backups.	Review `data/uploads` and backups; delete obsolete chats/documents; keep the model cache if you use RAG.

"Database is locked"	A second writer, antivirus scan, or a long transaction.	Run one app instance; wait; check the terminal. WAL and a five-second busy timeout handle normal contention.

FAQ

Can I run the classic bot and the hub at the same time?

Yes. They are independent transports in one process. The classic bot uses Baileys; hub agents use official tokens and OpenCode. They share the dashboard, SQLite, knowledge, and MCP server definitions, but not conversations or model sessions.

Can two hub agents share one official token?

No. One token maps to one active poller. Use a separate token for each agent/number. Sharing a token produces HTTP 409 and ambiguous ownership.

How many hub agents can I run?

There is no hard-coded agent limit. Capacity is bounded by the official API limits, model cost, OpenCode CPU/memory, and your ability to monitor the agents. Start with one, measure, then add.

How do I move a classic number to the hub?

Obtain an official token for the number, create a hub agent with the same prompt and knowledge, test it, then stop the classic poller and start the hub agent. Do not run both against the same number simultaneously.

Does deleting an agent delete its WhatsApp messages?

It removes the agent, token, bindings, skills, schedules, and sessions. Dashboard conversations and stored messages are separate records; delete them explicitly if required. Uploaded media remains in `data/uploads` until removed.

Where is the OpenCode config?

`opencode.json` in the project root. The hash of the managed file is in `data/opencode-managed.json`. Backups of an unmanaged file use `opencode.json.bak-<timestamp>`.

How do I rotate a token or API key?

For a hub token, update the agent's token and restart its poller. For OpenRouter, update Settings or `.env` and restart. Keep the old value until the new one is verified, then revoke it.

What if I lose `APP_SECRET`?

Stored secrets cannot be decrypted. Recreate `.env` with a new secret, then re-enter the OpenRouter key and every WhatsApp token. The message database and knowledge remain, but encrypted fields do not.

REFERENCE · APPENDIX A

Environment variables

The complete `.env` surface, with defaults and operational notes.

The app loads `.env` from the project root. Values already present in the process environment take precedence in normal Node deployments; keep one source of truth when running under a service manager.

VARIABLE	DEFAULT	PURPOSE AND NOTES
`PORT`	`8787`	Dashboard and API port.
`HOST`	`127.0.0.1`	Bind address. Change only with a reverse proxy and firewall plan.
`DATA_DIR`	`./data`	SQLite, sessions, uploads, model cache, and the managed config hash.
`ADMIN_PASSWORD`	none	Required for dashboard login. Use a long, unique value.
`APP_SECRET`	none	AES-256-GCM key material for stored secrets. 24+ random characters. Changing it invalidates existing encrypted values.
`OPENROUTER_API_KEY`	none	Classic model calls and the OpenCode child environment. Can also be set in the dashboard, which stores it encrypted.
`OPENROUTER_MODEL`	`openai/gpt-4.1-mini`	Classic default model. New hub agents inherit it with `openrouter/` added.
`OPENROUTER_BASE_URL`	`https://openrouter.ai/api/v1`	OpenAI-compatible endpoint. Change only for a compatible gateway.
`LOG_LEVEL`	`info`	Logger verbosity. `debug` is useful for embeddings, MCP, and runtime diagnosis.
`APP_URL`	`http://HOST:PORT`	Base URL used by the TUI. Set it when HOST/PORT differ from the defaults.
`OPENCODE_BIN`	`opencode`	Executable name or full path. Use the full path when the service manager's PATH differs from your shell.
`OPENCODE_PORT`	`4096`	Base port for the managed OpenCode server. It tries a few following ports if busy.
`WHATSAPP_AGENT_API_BASE`	`https://api.whatsapp.com/agent/v1`	Official Agent Platform base URL. Override only for a compatible test environment.
`VITE_API_BASE`	empty	Development-only API base for the Vite client. Leave empty for the same-origin proxy.
`NODE_ENV`	unset	`production` enables production-mode serving and logging behaviour. Set it under a process manager.

SECRET-HANDLING ORDER

Set `APP_SECRET` and `ADMIN_PASSWORD` before you store an OpenRouter key or any hub token. If you rotate `APP_SECRET`, re-enter every encrypted secret and update the backup set in the same maintenance window.

How values resolve

- `.env` is read at startup. A value already present in the process environment wins; restart after editing the file.
- An OpenRouter key saved in **Settings** is encrypted in SQLite and takes precedence over `OPENROUTER_API_KEY`. Delete it in Settings to fall back to the environment value.
- The classic model in **Settings** takes precedence over `OPENROUTER_MODEL`. Hub agents store their own model and do not follow this value after creation.
- `APP_SECRET` and `ADMIN_PASSWORD` have no dashboard override. Change them in `.env` and restart.
- `APP_URL` is used by the TUI and as the app's outbound referer; it is not a public tunnel.

Rotating a secret safely

1. **OpenRouter key:** create the replacement, save and test it in Settings, then update or remove the old environment value. Restart the app so the managed OpenCode child receives the new key.
2. **Hub token:** update the token on the agent, restart the poller, and send a real test message. Revoke the old token at the platform after the new one works.
3. **Admin password:** change `ADMIN_PASSWORD`, restart, and sign in with the new value. Existing dashboard sessions may remain valid until they expire.
4. **APP_SECRET:** stop the app, back up `.env` and `data/`, set the new secret, start, and re-enter every encrypted key and token. Old values cannot be recovered without the old secret.
5. **After any rotation:** update the backup set and confirm that a restore would include the matching secret.

Common configuration mistakes

MISTAKE	CONSEQUENCE	FIX
`APP_SECRET` changed casually	Stored keys and tokens become unreadable.	Restore the old secret or re-enter every secret; do not run with a mismatch.
Old dashboard key left in place	The environment key appears configured but is ignored.	Delete the dashboard key in Settings, then retest.
`OPENCODE_BIN` only on your shell PATH	The service manager starts the app but hub prompts fail.	Use the absolute path to `opencode.exe`.
`HOST=0.0.0.0` without TLS	The dashboard and admin API are exposed in plaintext.	Bind localhost or put a TLS reverse proxy and source restrictions in front.

REFERENCE · APPENDIX B

Scripts, ports, and data map

Where the commands, processes, and files live.

npm scripts

COMMAND	WHAT IT DOES
`npm run dev`	Starts the API (with watch mode) and the Vite dev server together.
`npm run dev:server`	API only, TypeScript watch mode.
`npm run dev:client`	Vite only; requires the API on port 8787.
`npm run build`	Builds the server to `dist/` and the dashboard to `public/admin/`.
`npm run build:server`	Server build only.
`npm run build:web`	Dashboard build only.
`npm start`	Runs the compiled server from `dist/index.js`.
`npm run tui`	Starts the terminal hub monitor against the running API.
`npm run typecheck`	Type-checks server and web without emitting files.
`npm test`	Runs the Node test suite in `test/`.
`npm run setup`	Runs the repository setup helper, if present.

Processes and ports

PROCESS	DEFAULT	STARTED BY
Express API + dashboard	`:8787`	`npm start` or `npm run dev:server`
Vite dev server	`:5173`	`npm run dev` / `dev:client`
Managed OpenCode	`:4096` +	First hub prompt or agent start
MCP stdio servers	child processes	Enabled MCP servers, on demand
Classic WhatsApp socket	outbound	App startup (Baileys)

Data directory

PATH	CONTENTS
`data/agent.sqlite`	Settings, chats, messages, runs, knowledge, MCP servers, hub agents/tokens, skills, schedules, sessions.
`data/agent.sqlite-wal` / `-shm`	SQLite WAL and shared-memory files. Include them in live copies or use a proper backup.
`data/wa-auth/`	Classic Baileys session credentials. Sensitive.
`data/models/`	Hugging Face/Transformers cache for the local embedding model.
`data/uploads/`	Inbound hub media. Not deleted when a chat is deleted.

`data/opencode-managed.json`	Hash and path of the generated OpenCode config.
`opencode.json`	Generated runtime config at the project root. Contains prompts and MCP settings, not WhatsApp tokens.
`opencode.json.bak-*`	One-time backup of an unmanaged config that existed before the hub took over.
`.opencode/skills/`	Generated skill directories for hub agents (`wa-hub-*`).
`.env`	Environment secrets. Never commit or share.

Generated OpenCode shape

The generated file follows this shape (values change with your agents):

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/openai/gpt-4.1-mini",
  "mcp": {
    "servers": {
      "filesystem": {
        "type": "local",
        "command": ["npx", "-y", "@modelcontextprotocol/server-filesystem", "./data"],
        "codemode": false
      }
    }
  },
  "agents": {
    "wa_hub_<id>": {
      "description": "WhatsApp agent: Customer Support",
      "mode": "primary",
      "model": "openrouter/openai/gpt-4.1-mini",
      "system": "...base prompt + agent prompt + skills...",
      "steps": 12,
      "permissions": [
        { "action": "filesystem_*", "resource": "*", "effect": "deny" }
      ]
    }
  }
}
```

The example deny block is what an *unbound* agent receives. A bound agent does not get the deny rules for that server. Do not copy this file into another project; regenerate it from the dashboard.

REFERENCE · APPENDIX C

Cron, models, and API limits

The small reference tables worth keeping next to the dashboard.

Cron examples

EXPRESSION	MEANING
`0 9 * * *`	Every day at 09:00.
`0 8 * * 1-5`	Weekdays at 08:00.
`30 17 * * 5`	Fridays at 17:30.
`0 */2 * * *`	Every two hours on the hour.
`15 7 1 * *`	The first day of each month at 07:15.
`0 9 * * 1`	Mondays at 09:00.

Five-field cron: minute, hour, day of month, month, day of week. Expressions are validated when saved. The timezone comes from **Settings**; an empty value uses the server's local zone.

Model identifiers

EXAMPLE	NOTES
`openrouter/openai/gpt-4.1-mini`	OpenRouter model with the required OpenCode provider prefix.
`openrouter/anthropic/claude-sonnet-4`	Provider family and model id.
`openrouter/google/gemini-2.5-flash`	Useful for fast, lower-cost replies.
`openrouter/nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free`	Free models can work for smoke tests; expect rate limits and variable tool use.

Use a model your OpenRouter key can access. Hub models are not read from `OPENROUTER_MODEL` after creation; edit the agent to change them.

Operational constants

CONSTANT	VALUE
Official API documented caps	12 sends, 12 status, 15 polls, 12 media per minute, per agent
Applied internal caps	11 sends, 11 status, 14 polls, 11 media per minute
Poll offset	64-bit value stored in SQLite
Hub message dedupe	`hub_messages` keyed by message id
Long reply split	About 3,500 characters per WhatsApp message
Hub media skip threshold	20 MB per inbound file
Dashboard upload limit	25 MB per file; supported text/document types
OpenCode agent steps	Up to 12 per prompt
Test prompt context	`test` session key

Schedule context	`schedule` session key
Customer context	One session per agent + sender id
Fallback reply	“Sorry, I could not handle that message right now. Please try again in a moment.”
Poll collision	HTTP 409 — another poller owns the token

Status codes you will see

CODE	MEANING IN THIS APP
400	Validation or a user-actionable start/test error (no token, disabled agent, bad prompt, rejected config).
401	Dashboard authentication required, or an official API token was rejected.
404	Record/route not found; for a stale OpenCode session, the runtime clears it and creates a new one.
409	Another poller owns the official token.
429	Rate limited by the provider or the app's login/API limiter.
500+	Unhandled server, runtime, or upstream failure. Read the log; do not keep retrying blindly.

Tool naming

MCP tools are exposed with a normalized server prefix. A server named `filesystem` produces names such as `filesystem_read_file`. Per-agent deny rules use `<server>_*` and also block OpenCode's generic resource listing/reading for unbound servers. If a tool is missing, check the server name, the binding, the enabled state, and the disabled-tools list before changing the model.

Glossary

The vocabulary used in the dashboard, the logs, and this guide.

TERM	MEANING
Agent	A configured WhatsApp identity: token, model, system prompt, MCP bindings, knowledge, skills, and schedules. In the hub, each agent has its own generated OpenCode entry.
Baileys	The unofficial WhatsApp Web library used by the classic bot. Requires QR linking and carries account-risk caveats.
Creator	The first WhatsApp sender recorded for a hub agent. Schedules and test sends target this person until the agent is deleted or the record changes.
Cron	A five-field schedule expression (minute, hour, day of month, month, day of week) used by hub schedules.
Embedding	A numeric representation of text used for semantic search. This app computes embeddings locally with `all-MiniLM-L6-v2`.
Fallback reply	The short WhatsApp message sent when the hub cannot obtain a model reply. It is a symptom, not a fix; check the logs.
Hub	The multi-agent mode that uses official WhatsApp Agent Platform tokens and a managed OpenCode runtime.
Knowledge	Documents or text stored in SQLite, chunked and embedded for retrieval. Attach a subset to each hub agent.
MCP	Model Context Protocol. A standard way to expose tools and resources to an agent over stdio, SSE, or Streamable HTTP.
Model provider	The service that performs inference. This deployment normally uses OpenRouter, which forwards to an upstream provider.
Offset	The official API cursor that tracks which messages have been consumed. Stored as a 64-bit value per agent.
OpenCode	The agent runtime used by the hub. The app spawns `opencode serve`, generates its config, and runs prompts through its HTTP API.
Poller	The per-agent loop that long-polls the official API for new messages. One active poller per token.
RAG	Retrieval-augmented generation: search relevant knowledge first, then include the passages in the model prompt.
Session	The model's conversation memory. Hub sessions are keyed by agent and sender, with separate test and schedule keys.
Skill	A reusable Markdown instruction block attached to an agent and written into its generated OpenCode prompt.
System prompt	The agent's standing brief: role, tone, constraints, and escalation rules. The runtime adds a WhatsApp-specific base prompt.
Token	In this guide, usually the official per-agent WhatsApp credential. Also used for the dashboard session cookie; context makes it clear.

Tool call	A model-requested action handled by an MCP server. Tools can read data or cause real-world effects; permission is an admin decision.
Transport	The path messages take: classic Baileys or the official WhatsApp Agent Platform.
TUI	The terminal UI started with `npm run tui` for monitoring and start/stop/reset actions.

Keep this guide with the deployment

Replace the placeholder branding when the kit is ready. Update the screenshots when the dashboard changes, and keep the environment table in sync with `.env.example`. A guide that describes a system you no longer run is worse than no guide at all.

WhatsApp Agent · User & Administrator Guide · Edition 1.0 · October 2026 · Placeholder brand.

WhatsApp is a trademark of WhatsApp LLC. This project is not affiliated with WhatsApp, Meta, OpenRouter, or OpenCode. Model names and platform behaviour belong to their respective owners and may change.